

**ДЕРЖАВНИЙ УНІВЕРСИТЕТ
ІНФОРМАЦІЙНО-КОМУНІКАЦІЙНИХ ТЕХНОЛОГІЙ
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ
ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ**

КУРСОВА РОБОТА

на тему:

«Розробка гри "Endless Hallway: Asylum" у жанрі хоррор з елементами пошуку
аномалій»

з дисципліни:

«Об'єктно-орієнтоване програмування C#»

студента 2 курсу групи ПД-22

Кафедри інженерії програмного забезпечення

Тараканова Родіона Сергійовича

Викладач

асистент кафедри

Інженерії програмного забезпечення,

Ярошевський Олександр Вікторович

оцінка _____

Київ 2026

РЕФЕРАТ

Курсова робота, 67 с., 34 рис., 3 додатки, 11 джерел.

Ключові слова: тривимірний ігровий застосунок, Unity, C#, компонентно-орієнтована архітектура, MonoBehaviour, психологічний хоррор, лімінальні простори, аномалії, 3D Spatial Audio, PlayerPrefs.

Об'єктом дослідження було: процес проектування, архітектурної організації, алгоритмічної оптимізації та програмної реалізації тривимірних інтерактивних систем та відеоігор на базі компонентно-орієнтованого підходу.

Метою дослідження було: проектування та практична розробка тривимірного ігрового застосунку «Endless Hallway: Asylum» у жанрі психологічного інді-хоррору з використанням об'єктно-орієнтованого програмування, мови C#, кросплатформного двигуна Unity та конвеєра рендерингу URP, а також поглиблення теоретичних знань та набуття практичних навичок у побудові компонентно-орієнтованої архітектури, розробці скінченних автоматів для керування станами гри та інтеграції систем просторового 3D-аудіо.

В ході дослідження були одержані: функціональний тривимірний ігровий застосунок «Endless Hallway: Asylum», який надає користувачам можливість інтерактивної взаємодії з оточенням від першої особи, реалізує унікальний ігровий цикл перевірки уважності на базі 9 поверхів-ітерацій, містить механіку синусоїдального погойдування камери, кастомне меню паузи та систему динамічного перепризначення клавіш. Розроблено гнучку архітектуру менеджерів стану, що забезпечує випадкову підміну об'єктів, зміну масштабу та активацію ізольованих тригерних скрімерів. Реалізовано комплексне інженерне рішення з оптимізації

освітлення та інтегровано просторові звукові зони зливи й оточення за допомогою 3D Audio. Побудовано UML-діаграми діяльності, послідовності та блок-схеми, що описують алгоритми валідації вибору шляху гравцем та логіку очищення сцени від аномалій.

ЗМІСТ

СЛОВНИК ТЕРМІНІВ.....	6
ВСТУП.....	9
1 ХАРАКТЕРИСТИКА ТА АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ	11
1.1 Історія виникнення та еволюція жанру психологічних інді-хоррорів	11
1.2 Концепція лімінальних просторів та механіки спостереження в іграх	12
1.3 Аналіз існуючих аналогічних програм	14
2 ПРИЗНАЧЕННЯ ТА СТРУКТУРА ПРОГРАМНОГО ПРОДУКТУ	16
2.1 Специфікація вимог до створення.....	16
2.1.1 Функціональні вимоги.....	16
2.1.2 Нефункціональні вимоги.....	17
2.1.3 Вимоги користувача	18
2.2 Можливі ризики для проєкту	19
2.3 Цільова аудиторія.....	20
3 МОДЕЛЮВАННЯ ТА ПРОЄКТУВАННЯ ІГРОВОЇ СИСТЕМИ	22
3.1 Функціональна модель системи.....	22
3.2 Архітектурні рішення	23
3.2.1 Вибір архітектури.....	23
3.2.2 Загальна структура ігрового проєкту.....	24
3.2.3 Архітектурна організація менеджерів стану	26
3.2.4 Діаграма послідовності для генерації етажу	26
3.2.5 Діаграма діяльності системи.....	27
3.3 Алгоритм роботи розумного очищення та активації аномалій	28
3.4 Вибір та обґрунтування підходу до локального збереження даних	29
4 ПРОЄКТУВАННЯ СТРУКТУРИ ПРОГРАМИ	31
4.1 Обґрунтування моделі розробки ігрового застосунку	31
4.2 Середовище розробки та технічний стек.....	31

4.3 Вбудовані підсистеми двигуна	32
4.4 Побудова програмного рішення	33
4.4.1 Логіка випадкової підміни об'єктів та умов перемоги	34
4.4.2 Програмування контролера та 3D-аудіо у вікнах	35
5 Тестування.....	38
5.1 Що таке тестування в GameDev і навіщо воно потрібне?	38
5.2 Тестування інтерфейсу користувача та навігації меню	39
5.2.1 Головне меню та налаштування аудіопараметрів	39
5.2.2 Тестування меню паузи в ігровому процесі.....	43
5.3 Тестування відсікання тригерів скрімерів на чистому колі	45
5.4 Тестування зміни аудіокліпів вікон та блокування паузи при екрані перемоги	46
Висновки	48
ПЕРЕЛІК ПОСИЛАНЬ	50
ДОДАТОК А. ДЕМОНСТРАЦІЙНІ МАТЕРІАЛИ	51
ДОДАТОК Б. РЕПОЗИТОРІЙ ПРОГРАМНИХ МОДУЛІВ.....	59
ДОДАТОК В. ЛІСТИНГИ ПРОГРАМНИХ МОДУЛІВ	59

СЛОВНИК ТЕРМІНІВ

ІД	Комплексне програмне забезпечення, що надає середовище для розробки тривимірних інтерактивних застосунків, обробки фізики, рендерингу та звуку.
БД	База даних; впорядкований набір структурованої інформації.
ПЗ	Програмне забезпечення.
ГПВПЧ	Генератор псевдовипадкових чисел; алгоритм, що генерує послідовність чисел, яка задовольняє вимогам випадковості. Використовується в AnomalyLogic для розрахунку ймовірності спавну аномалій (50%).
API	Прикладний програмний інтерфейс; набір готових класів, процедур і функцій, що надаються двигуном Unity для взаємодії з апаратною частиною.
Unity	Кросплатформний високопродуктивний ігровий двигун від компанії Unity Technologies, обраний як базовий інструментарій для розробки тривимірного хоррору.
MonoBehaviour	Базовий клас у Unity, від якого наслідуються всі створені C#-скрипти, що дозволяє прив'язувати код до ігрових об'єктів та використовувати життєвий цикл подій.
Component-Based	Компонентно-орієнтований підхід до програмування; архітектурний патерн, за якого функціонал об'єкта розширюється шляхом додавання до нього ізольованих компонентів.
3D Spatial Audio	Технологія просторового тривимірного звуку; метод обробки аудіопотоку, за якого амплітуда та баланс звуку динамічно змінюються залежно від відстані та кута між гравцем та джерелом.

PlayerPrefs	Вбудований у Unity клас для локального збереження невеликих обсягів даних між ігровими сесіями; використовується для персистенції лічильника поверхів.
Raycasting	Метод геометричного проектування променя у тривимірному просторі з метою виявлення перетину з колізійними поверхнями об'єктів. Використовується для механіки взаємодії з дверима на відстані interactDist.
Dot Product	Скалярний добуток двох векторів; математична операція, що використовується у коді дверей для визначення кута погляду гравця відносно площини дверей.
Coroutine	Функція в C#, яка здатна призупиняти своє виконання і відновлювати його через заданий проміжок часу. Використовується для таймінгів скримерів та миготіння ламп.
Mixed Lighting	Гібридний режим освітлення в Unity, за якого статичні об'єкти отримують запечені карти світла, а динамічні продовжують відкидати тіні в реальному часі.
Light Probes	Світлові зонди; технологія, яка фіксує інформацію про запечене світло у просторі та передає її динамічним об'єктам для їхньої коректної візуальної інтеграції зі статичним оточенням.
Stuttering	Візуальний ефект мікро-фризів або нерівномірності виведення кадрів, викликаний різкими скачками часу прорахунку кадру на графічному конвеєрі.
Shadow Atlas	Текстурний атлас тіней; велика комбінована текстура, в яку Unity пакує карти тіней від усіх активних джерел світла в поточному кадрі.
Head Bobbing	Геймдизайнерський алгоритм синусоїдального коливання камери по вертикальній та горизонтальній осях, що імітує біомеханіку рухів людини під час ходьби або бігу від першої особи.

IDE	Інтегроване середовище розробки програмного забезпечення; у межах проєкту — Microsoft Visual Studio, що використовується для написання та відладки коду на C#.
UML	Уніфікована мова моделювання; набір графічних діаграм для специфікації та документування логічної структури та поведінки ігрових менеджерів.
UX/UI	Користувацький досвід та користувацький інтерфейс. Включає ергономіку керування персонажем та графічне оформлення меню паузи, головного меню та панелі перемоги.
Smart Cleanup	Алгоритм розумного архітектурного очищення сцени, реалізований у AnomalyLogic, який при завантаженні нового кола примусово обнуляє і деактивує всі аномальні тригери.

ВСТУП

Актуальність теми. Індустрія тривимірних інтерактивних застосунків та відеоігор на сучасному етапі розвитку демонструє стрімке зміщення акцентів від класичних екшен-механік у бік психологічного занурення та ігор, заснованих на спостережливості. Особливе місце в цьому сегменті посіли інді-хоррори, що використовують феномен лімінальних просторів та концепцію «петлі ходьби» (walking simulator / anomaly detection). Успіх таких комерційних проєктів, як The Exit 8, довів високу затребуваність ігор, де головним інструментом геймплею є не фіз взаємодія чи бойова система, а когнітивний аналіз змін у статичному середовищі.

Розробка ігрового застосунку «Endless Hallway: Asylum» є актуальною з науково-практичної точки зору, оскільки вона досліджує принципи побудови алгоритмів процедурної або випадкової модифікації тривимірних сцен, оптимізацію систем локального збереження даних в умовах циклічних перезавантажень, а також архітектурне проектування менеджерів стану гри на базі компонентно-орієнтованого підходу.

Для досягнення поставленої мети необхідно вирішити такі завдання:

1. проаналізувати історію розвитку жанру психологічних інді-ігор та теоретичний базис концепції лімінальності;
2. сформулювати повний комплекс функціональних, нефункціональних вимог та вимог користувача до розроблюваного програмного продукту;
3. оцінити архітектурні ризики, пов'язані з витоками пам'яті та конфліктами ресурсів у межах сцени;
4. розробити архітектуру взаємодії менеджерів станів гри та контролерів користувацького введення;
5. реалізувати алгоритм випадкової підміни об'єктів, динамічного масштабування та ізольованої ініціалізації скриптів-скримерів;
6. провести комплексне тестування системи збереження ітерацій поверхів за допомогою механізму PlayerPrefs та верифікувати коректність роботи тригерів

відсікання аномалій.

Предмет дослідження. Алгоритми випадкової генерації станів сцени, методи обробки просторового звуку, інтеграція кастомного введення та механізми керування станами циклічного ігрового процесу.

1 ХАРАКТЕРИСТИКА ТА АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Історія виникнення та еволюція жанру психологічних інді-хоррорів

Індустрія розробки цифрових розважальних продуктів за останні три десятиліття пройшла масштабний шлях трансформації, де жанр інтерактивних жахів (Survival Horror) зазнав чи не найбільших концептуальних змін. Історично витoki жанру базувалися на жорстко детермінованих механіках, де гравцеві протиставилися явні фізичні загрози, а керування здійснювалося за допомогою фіксованих ракурсів віртуальної камери (як у класичних серіях Resident Evil, Silent Hill або Alone in the Dark). Технологічні обмеження того часу змушували розробників фокусуватися на дефіциті ресурсів та складному менеджменті інвентарю. Користувач у таких системах виступав як активний бойовий елемент, що протистоїть агресивному середовищу за допомогою доступного арсеналу зброї.

Однак із розвитком індустрії та перенасиченням ринку однотипними екшен-хоррорами, фокус інтересу аудиторії почав зміщуватися у бік інді-розробників, які отримали доступ до потужних безкоштовних двигунів. Це призвело до виникнення концептуально нової хвилі психологічних жахів від першої особи. Першим фундаментальним проривом став реліз проекту Amnesia: The Dark Descent (2010 р.) від студії Frictional Games, а згодом — Outlast (2013 р.) від Red Barrels [10]. Головною інженерною та геймдизайнерською новацією цих продуктів стало повне позбавлення гравця можливостей фізичного захисту чи нападу. Користувач перетворився на безпорадного спостерігача, чийми єдиними механіками взаємодії стали втеча, переховування та використання обмежених джерел світла. Це кардинально змінило психологічний профіль залучення: страх перед зменшенням шкали здоров'я змінився глибоким ментальним тиском і почуттям перманентної вразливості [11].

Справжню революцію в контексті просторової організації ігрового процесу здійснив вихід інтерактивного тизеру P.T. (Playable Teaser, 2014 р.) за авторством Хідео Кодзіми [10]. Проєкт запропонував інноваційну архітектурну концепцію

заикленого коридору. Гравець опинявся в замкненому, побутовому L-подібному коридорі звичайного житлового будинку. Кожен перехід крізь фінальні двері не виводив користувача на новий рівень, а повертав його у вихідну точку того самого коридору. Проте з кожною новою ітерацією простір піддавався ледве помітним, але дедалі більш лякаючим трансформаціям: змінювався спектр освітлення, з'являлися сторонні звуки, переміщувалися побутові предмети, порушувалася геометрія стін. Сама ідея того, що знайоме, безпечне побутове середовище деформується за невідомими законами, заклала підвалини для формування окремого піджанру.

Сучасний етап розвитку ознаменувався виходом піджанру на рівень мікро-хоррорів, що базуються на побутовій рутині та максимальному реалізмі середовища [11]. Ці проєкти довели, що для створення високого рівня напруги не обов'язково моделювати масштабні готичні замки чи космічні станції. Достатньо помістити гравця в умови повсякденності та застосувати алгоритми випадкової модифікації середовища. Таким чином, еволюція жанру пройшла шлях від винищення монстрів у відкритому просторі до детального спостереження за статичним коридором, де найменша зміна є ознакою смертельної небезпеки [9].

1.2 Концепція лімінальних просторів та механіки спостереження в іграх

В основі візуального, атмосферного та структурного проєктування тривимірного простору гри «Endless Hallway: Asylum» покладено архітектурно-психологічний феномен лімінальності. Етимологічно термін походить від латинського слова *limen*, що перекладається як «поріг» або «перехідний стан» [6]. У культурології, соціології та архітектурі лімінальними просторами називають транзитні, проміжні локації, які не мають самостійної цінності й існують виключно для того, щоб людина перебувала в них тимчасово, здійснюючи перехід з однієї точки в іншу. Типовими прикладами таких просторів є довгі готельні коридори, підземні пішохідні переходи, порожні зали очікування аеропортів, ліфтові холи, багаторівневі паркінги та лікарняні рекреації у позаробочий час [7].

Коли лімінальний простір позбавляється своєї першочергової утилітарної

функції, у людській психіці виникає глибокий когнітивний дисонанс. Мозок розпізнає геометрію об'єктів як знайому і буденну, але відсутність звичного контексту та маркерів життєдіяльності викликає відчуття відчуженості, тривоги й підсвідомого очікування загрози. Цей ефект у сучасній когнітивній психології часто пов'язують із концепцією «зловісної долини», екстраполірованою на архітектурне оточення: простір виглядає «майже правильно», але саме це «майже» змушує відчувати небезпеку [8].

Імплементация концепції лімінальності в геймдеві за допомогою інструментів Unity дозволяє розв'язати низку важливих інженерних та оптимізаційних завдань:

економія обчислювальних ресурсів та оптимізація рендерингу: Замість проектування відкритих світів із мільйонами полігонів, розробник концентрує зусилля на обмеженій геометрії. Це дозволяє використовувати високодеталізовані текстури, налаштовувати запікання світла з максимальною точністю та впроваджувати складні шейдери матеріалів без втрати загальної продуктивності системи[2];

Керування увагою користувача: у великих ігрових просторах увага гравця розпоршується. У лімінальному коридорі кожен об'єкт потрапляє в поле зору. Це дозволяє геймдизайнеру перетворити саме оточення на головного опонента гравця[10].

Механіка спостереження повністю переформатовує класичний ігровий досвід. Гравець перестає бути механічним виконавцем квестів — він стає валідатором статичних даних [9]. При першому проходженні рівня користувач формує в пам'яті ментальний еталонний образ локації. На кожному наступному поверсі система випадковим чином активує або не активує аномалії. Завдання гравця — порівняти поточний візуальний та звуковий стан сцени з еталоном. Якщо пам'ять підказує, що двері туалету раніше були зачинені, а зараз вони прочинені, або якщо інвалідний візок змістився на кілька сантиметрів, гравець повинен прийняти логічне рішення: розвернутися й піти в протилежний бік, сигналізуючи системі про виявлення помилки в структурі простору.

1.3 Аналіз існуючих аналогічних програм

Для забезпечення конкурентоспроможності розроблюваного програмного продукту та формування чіткої стратегії його архітектурного проектування було виконано глибокий деконструкційний аналіз та порівняння існуючих на ринку комерційних програм-аналогів. Основними об'єктами дослідження виступили ігри The Exit 8 та проекти лінійки I'm on Observation Duty [11].

The Exit 8 (розробник KOTAKE CREATE) побудований на базі ігрового двигуна Unreal Engine 5. Проєкт демонструє високий рівень фотореалізму завдяки використанню технологій динамічного освітлення. Гравець переміщується по підземному переходу метрополітену, шукаючи відхилення від норми. Головним обмеженням даного аналога є його жорстка детермінованість: геометрія коридору абсолютно симетрична, налаштування керування обмежені базовою розкладкою клавіш без можливості кастомізації під індивідуальні потреби користувача. Крім того, аудіосистема проєкту є переважно статичною: звуки кроків та гудіння ламп мають фіксовану амплітуду і не адаптуються до динамічних погодних змін середовища.

Проєкти серії I'm on Observation Duty використовують ігровий двигун Unity, але пропонують гравцеві перспективу через статичні камери відеоспостереження (CCTV). Користувач перемикається між дискретними ракурсами кімнат і заповнює текстові репорти при виявленні аномалій. Такий підхід суттєво знижує так званий «ефект присутності» та іммерсивність, перетворюючи тривимірну гру на двовимірний пошук відмінностей на статичних зображеннях [13].

Розроблюваний застосунок «Endless Hallway: Asylum» інтегрує в собі найкращі архітектурні та геймдизайнерські рішення, усуваючи обмеження аналогів. Порівняльні технічні характеристики наведено у таблиці 1.3.

Особливу увагу в концепції приділено створенню глибокої атмосфери хорору через кастомні тригери, просторове аудіо та динамічну зміну оточення лікарні. Таким чином, деконструкція ринкових аналогів дозволила сформувати чітке технічне завдання для створення оптимізованого, іммерсивного та конкурентоспроможного інді-продукту.

Таблиця 1.3

Порівняльна таблиця аналогів

Параметри порівняння та технічні критерії	The Exit 8 (KOTAKE CREATE)	I'm on Observation Duty	«Endless Hallway: Asylum» (Розроблюваний проєкт)
Базовий інструментарій / Двигун	Unreal Engine 5	Unity 2021/2022	Unity 6 (6000.x.x) / C#
Режим перспективи камери (View)	Від першої особи (First-Person View)	Дискретні статичні камери (CCTV)	Від першої особи + кастомний Head Bobbing
Умова успішної фіналізації сесії	Досягнення виходу №8	Виживання до 06:00 (таймер)	Досягнення 9-го кола (CurrentLoop >= 9)
Алгоритм валідації вибору шляху	Фізичний розворот на 180°	Надсилення текстової форми	Вибір вектора руху: Вперед або назад
Типи просторових аномалій	Візуальні підміни мешів	Зникнення, спавн, привиди	Комплексні: Swap, Scale, тригерні скринери
Динамічна аудіосистема	Відсутня (статичний ембієнт)	Дискретні шуми перешкод	Прогресивне 3D Audio з симуляцією зливи
Кастомізація введення (Input)	Фіксована (Hardcoded)	Відсутня	Динамічний KeyBindManager (Rebinding)

Програмний продукт «Endless Hallway: Asylum» виграє за рахунок поєднання динаміки від першої особи з розширеним спектром типів аномалій. Особливе місце посідає інтеграція тригерних інтерактивних об'єктів[1]. На відміну від аналогів, гра містить розвинену систему підсистем користувача: меню паузи, яке повністю ізолює ігровий час [1], та гнучкі налаштування графіки й звуку[3], що забезпечує високий рівень адаптивності програмного продукту до кінцевого користувача[6].

2 ПРИЗНАЧЕННЯ ТА СТРУКТУРА ПРОГРАМНОГО ПРОДУКТУ

2.1 Специфікація вимог до створення

Планування програмного забезпечення є одним із найважливіших етапів розробки, який передбачає формування чіткого сценарію роботи та архітектурної структури майбутньої системи. У процесі планування проводиться ґрунтовний аналіз вимог, що дозволяє визначити ключові функції продукту, уточнити спосіб взаємодії користувача з інтерфейсом та гарантувати, що кінцевий результат відповідатиме всім заданим критеріям якості. Вимоги визначаються в процесі аналізу предметної області та фіксуються в специфікації. Для розроблюваної системи кадрового обліку «Staff Flow» вимоги поділяються на функціональні, нефункціональні та вимоги користувача[10].

2.1.1 Функціональні вимоги

Функціональні вимоги визначають перелік функцій, алгоритмів та логічних реакцій, які програмна система зобов'язана виконувати після її реалізації.

1. Керування прогресом та ігровими циклами: програмна система повинна забезпечувати збереження, зчитування та модифікацію індексу поточного кола коридору. Запис даних про прогрес має здійснюватися в енергонезалежну пам'ять цільового пристрою щоразу після фіксації вибору гравця на поточному поверсі, щоб унеможливити втрату прогресу при раптовому закритті програми.

2. Генерація та адміністрування аномальних станів: на етапі ініціалізації кожної нової ітерації локації алгоритм системи повинен обчислювати випадкову величину для визначення появи аномалії за заздалегідь встановленою ймовірністю. При виборі конкретного аномального об'єкта система зобов'язана автоматично блокувати та деактивувати всі інші альтернативні об'єкти для запобігання їхнього взаємного накладання.

3. Валідація просторового вибору користувача: архітектура системи має

передбачати відстеження напрямку руху персонажа на краях локації. При наявності згенерованої аномалії спроба пройти вперед через фінальну точку має розпізнаватися як помилка з подальшим скиданням лічильника кіл до нуля; повернення назад у стартову зону за наявності аномалії має валидуватися як правильний вибір, що веде до збільшення лічильника кіл на одиницю.

4. Обробка інтеракцій та хоррор-механік: логіка взаємодії з дверима повинна забезпечувати зміну їхнього стану за сигналом від пристроїв введення, враховуючи задані межі дистанції та напрямок погляду гравця. У разі активації специфічної аномалії, відкриття дверей має ініціювати запуск каскаду подій з наступним автоматичним вивантаженням цих ресурсів із пам'яті за таймером.

5. Керування фіналом ігрової сесії: при досягненні лічильником кіл цільового переможного значення система повинна примусово заблокувати введення з пристроїв керування для контролера персонажа, зупинити прорахунок ігрового часу та фізики, вивести на екран графічний інтерфейс перемоги, а також перевести курсор миші у розблокований стан для взаємодії з меню.

2.1.2 Нефункціональні вимоги

Нефункціональні вимоги встановлюють критерії якості, технічні обмеження, стандарти оптимізації та системні вимоги до апаратного забезпечення:

1. продуктивність та плавність візуального ряду: час рендерингу одного кадру не повинен перевищувати 16.6 мілісекунд, що має гарантувати стабільну частоту кадрів не менше 60 FPS на персональних комп'ютерах середнього класу. Процеси динамічного завантаження об'єктів або активації сценаріїв аномалій не повинні викликати мікрофризів;

2. ефективність управління оперативною пам'яттю (Memory Management): оскільки ігровий процес планується базувати на циклічному перезавантаженні ігрового простору, архітектура менеджменту ресурсів має виключати накопичення застарілих даних в RAM/VRAM. Усі динамічно створені матеріали, текстури та аудіопотоки повинні примусово очищатися, а логіка взаємодії скриптів має виключати появу «висячих» посилань для запобігання витокам пам'яті;

3. оптимізація графічного конвеєра: для забезпечення стабільної продуктивності запікання карт освітлення для статичної геометрії має проводитися попередньо, на етапі розробки. Кількість викликів отрисовки у реальному часі має бути мінімізована шляхом обмеження радіусу прорахунку динамічних тіней від джерел світла;

4. сумісність та апаратні системні вимоги: програмний продукт повинен стабільно функціонувати на персональних комп'ютерах під керуванням ОС Windows 10/11 та відповідати наступним апаратним конфігураціям:

5. мінімальні системні вимоги (для гри у 720p / 30 FPS): процесор Intel Core i3-4160 / AMD FX-6300; 8 ГБ оперативної пам'яті; відеокарта Nvidia GeForce GTX 750 Ti / AMD Radeon R7 260X; 2 ГБ відеопам'яті;

6. рекомендовані системні вимоги (для гри у 1080p / 60 FPS): процесор Intel Core i5-7400 / AMD Ryzen 3 1200; 12 ГБ оперативної пам'яті; відеокарта Nvidia GeForce GTX 1050 Ti / AMD Radeon RX 560; 4 ГБ відеопам'яті;

7. локалізація та мовна сумісність: текстовий графічний інтерфейс, елементи головного меню, підказки та дебаг-логи консолі повинні проектуватися українською мовою із забезпеченням коректного відображення шрифтів кирилиці.

2.1.3 Вимоги користувача

Вимоги користувача описують взаємодію людини з програмним інтерфейсом, систему навігації та ергономіку керування:

1. механіка переміщення від першої особи: користувач повинен мати можливість інтуїтивного переміщення у тривимірному просторі за допомогою осей введення (клавіші W, A, S, D або стрілочки)[9]. Погляд камери має слідувати за рухом миші без затримок. Для підвищення реалістичності в скрипт PlayerMovement інтегрується математичний алгоритм Head Bobbing, який розраховує коливання камери по синусоїдальній траєкторії[11].

2. ергономіка взаємодії: при наведенні центрального перехрестя прицілу на інтерактивний об'єкт на відстані не більше 4 метрів, гравець активує дію натисканням клавіші E. Система повинна ігнорувати натискання, якщо користувач стоїть спиною

до об'єкта. Це реалізується за допомогою перевірки скалярного добутку вектора погляду персонажа та вектора напрямку на об'єкт, який має бути більшим за порогове значення 0.6[6];

3. головне меню: стартовий екран застосунку, що містить кнопки «Почати гру», «Налаштування» та «Вихід з гри»[10];

4. меню паузи: активується в будь-який момент геймплею при натисканні клавіші Escape. Воно має повністю заморожувати ігровий процес, відкривати доступ до повзунків гучності звуку [3] та можливості перепризначення клавіш керування через KeyBindManager[1];

5. візуальні індикатори прогресу: настінні таблички або цифрові індикатори ліфта мають оновлюватися динамічно, відображаючи поточний верифікований номер поверху, на якому перебуває користувач.

2.2 Можливі ризики для проєкту

Проєктування та розробка комп'ютерної гри у жанрі психологічного хоррору із циклічною структурою супроводжується специфічними геймдизайнерськими, ринковими та виробничими ризиками. Нижче наведено детальний аналіз трьох основних ризиків інтелектуального продукту та методи їхнього усунення, закладені в концепцію розробки:

1. ринковий ризик високої конкуренції та вторинності геймплею ігри, засновані на механіці спостереження за аномаліями та циклічних, представлені на ринку у великій кількості[11]. Існує висока ймовірність того, що гра «не вистрілить» через відсутність унікальності. Для нейтралізації цього ризику в архітектуру проєкту закладено унікальну перевагу — глибокий психологічний фокус, акцент на інтелектуальному протистоянні з переслідувачем та кастомні скрипти динамічної зміни атмосфери. Це дозволяє виділити гру серед аналогів не просто візуальними скримерами, а наростаючим саспенсом[1];

2. геймдизайнерський ризик монотонності та втрати інтересу гравця оскільки одна й та сама локація завантажується десятки разів поспіль, виникає серйозний

ризик швидкого вигорання користувача. Якщо аномалії будуть повторюватися або виявляться занадто очевидними, гравець втратить почуття страху, і ігровий процес перетвориться на рутину[6]. Для усунення цього ризику в логіку менеджера AnomalyLogic закладено алгоритм випадкової генерації станів сцени та розраховано оптимальний баланс складності. Чергування явних скримерів із тонкими, ледь помітними змінами в оточенні підтримує постійну ментальну напругу гравця та мотивує його доходити до фінального дев'ятого кола[9];

3. виробничий ризик «роздування масштабів» та зриву дедлайнів: у процесі розробки інді-ігор часто виникає бажання нескінченно покращувати графіку, додавати нові 3D-моделі чи ускладнювати механіки, що веде до дефіциту часу на фінальне тестування та полірування продукту[3]. Це загрожує релізом «сирої» версії з багами. Цей ризик нівелюється шляхом жорсткого модульного проектування. Логіку взаємодії розроблено в лаконічному та оптимізованому вигляді без зайвих нашарувань анімацій. Використання чіткого переліку готових ассетів дозволяє здати стабільний робочий білд у суворо встановлені терміни[2].

2.3 Цільова аудиторія

Цільовий сегмент споживачів розроблюваного програмного продукту — це користувачі віком від 16 років, які активно цікавляться жанрами психологічних детективів, симуляторів ходьби, інтерактивних квестів та ігор на уважність[11]. Дана аудиторія характеризується високими вимогами до атмосфери продукту, деталізації звукового оточення та логічної зв'язності ігрового світу. Вони шукають у грі не механічне випробування швидкості реакції, а інтелектуальний виклик, заснований на концентрації уваги, аналізі середовища та роботі з власною пам'яттю[10].

З точки зору психології сприйняття хорор-елементів, проєкт «Endless Hallway: Asylum» відмовляється від концепції банальних і регулярних раптових переляків, замінюючи їх інструментами глибокого ментального тиску[11]:

1. маніпуляція упередженням очікування: найбільший рівень стресу людська психіка відчуває не в момент безпосереднього контакту з небезпекою, а в процесі її

напруженого очікування. Знаючи, що ймовірність появи аномалії на поверсі становить 50%, гравець самостійно заганає себе в стан підвищеної тривожності[10]. Він починає підозрювати кожен елемент декору, прискіпливо вглядатися в тіні й аналізувати кожен шурхіт, що максимізує психологічне занурення;

2. руйнування сталості об'єктів: дитяча та доросла психіка базово спирається на аксіому, що предмети матеріального світу стабільні: якщо шафа стоїть праворуч, вона не може стати більшою чи зміститися сама по собі. Коли гравець виявляє аномалію типу ScaleObject, це викликає миттєвий підсвідомий дискомфорт і страх, оскільки руйнуються звичні закони логіки та фізики віртуального світу[9];

3. контрастне акустичне кодування: постійний, монотонний і низькочастотний шум сильної зливи за вікнами лікарні виконує роль психологічного «білого шуму». Він частково маскує інші звуки і змушує гравця прислухатися. Коли на фоні цього монотонного шуму раптово спрацьовує скрипт ToiletDoorScreamer або лунає різкий гуркіт від колісниць, це викликає потужний старт-рефлекс на фізіологічному рівні — різкий стрибок пульсу та викид адреналіну, що забезпечує максимальний ефект хорору[3].

3 МОДЕЛЮВАННЯ ТА ПРОЄКТУВАННЯ ІГРОВОЇ СИСТЕМИ

3.1 Функціональна модель системи

Проектування тривимірного інтерактивного застосунку в жанрі психологічного хоррору вимагає детального опису процесів взаємодії між користувачем та віртуальним середовищем лікарні. На відміну від традиційних систем, де взаємодія має лінійний характер «введення-реакція», у грі «Endless Hallway: Asylum» функціональна модель працює у замкненому контурі зі зворотним зв'язком, що базується на когнітивному аналізі простору[10].

Користувач взаємодіє з системою через три основні підсистеми:

1. підсистема просторової навігації (PlayerMovement): транслює фізичні натискання клавіш керування у векторні зміщення персонажа в межах тривимірних координат сцени[9]. Одночасно з цим обчислюються амплітудно-частотні параметри Head Bobbing для забезпечення ефекту присутності;
2. підсистема фізичної взаємодії (Raycasting Interaction): кожну ітерацію оновлення кадру генерує колізійний промінь з центру камери гравця[4]. При перетині променя з об'єктами класів SimpleDoor або ToiletDoorScreamer на відстані, що не перевищує константу interactDist, система очікує активації керуючого сигналу;
3. підсистема логічного вибору (Loop Decision): взаємодіє з тригерами переходу сцени. Гравець здійснює фізичне переміщення вглиб коридору або повертається назад до ліфтової шахти, що запускає внутрішні обчислювальні процеси валідації стану поверху(див. рис. 3.1)[5].

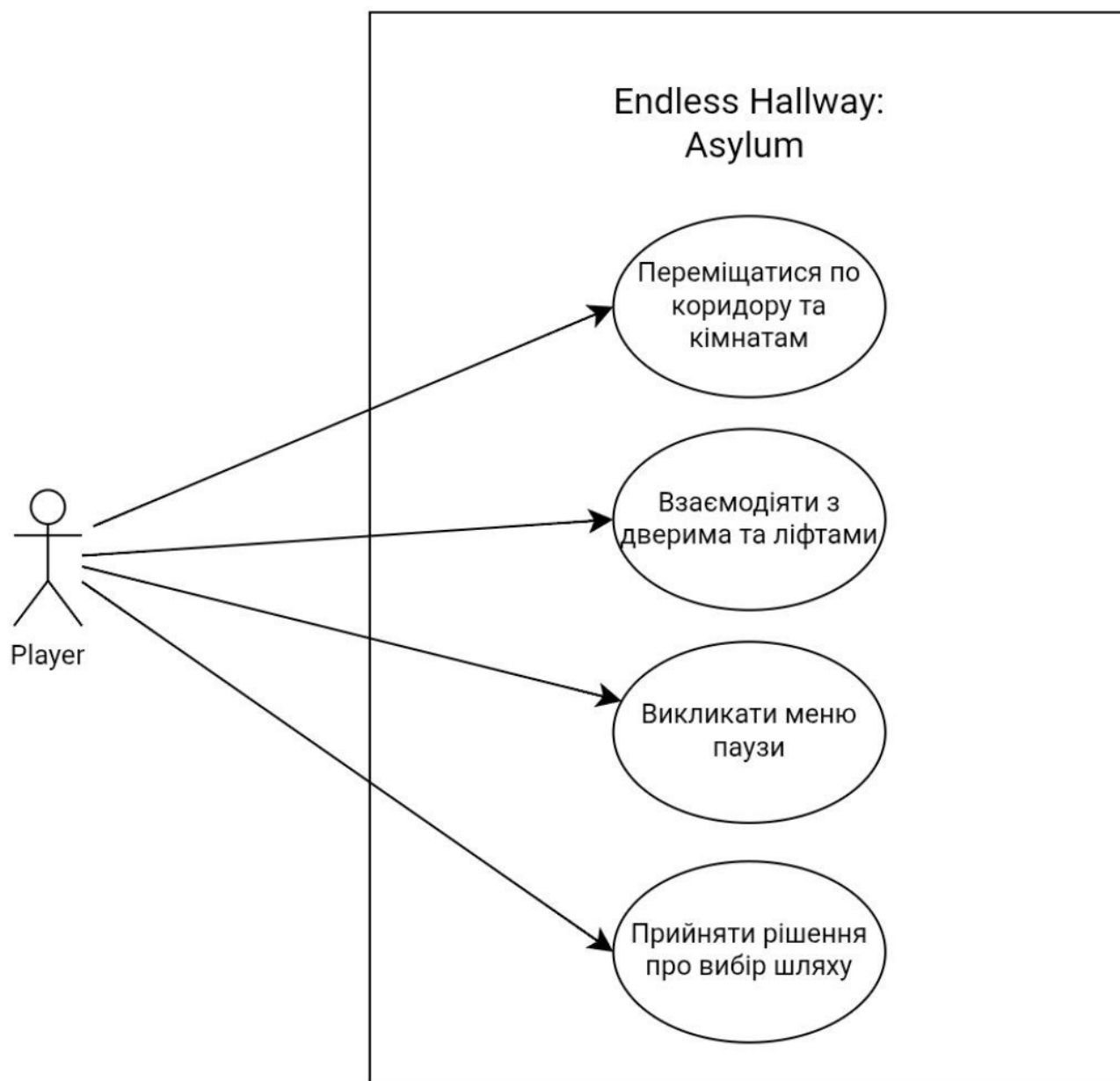


Рис. 3.1 Діаграма прецедентів гравця

3.2 Архітектурні рішення

3.2.1 Вибір архітектури

Під час проєктування програмного комплексу було відкинута класична монолітна архітектура на користь компонентно-орієнтованого патерну, який є фундаментальним для ігрового двигуна Unity[1]. Усі сутності на сцені є порожніми контейнерами, функціонал яких визначається набором підключених до них програмних компонентів, що є спадкоємцями базового класу `MonoBehaviour`[1].

Переваги вибору даного архітектурного підходу для хоррор-гри:

1. модульність та слабка зв'язність (Loose Coupling): скрипт ToiletDoorScreamer не керує логікою появи зомбі безпосередньо в своєму тілі — він посилається на готовий GameObject zombie. Це дозволяє гнучко замінювати тривимірну модель ворога в інспекторі Unity без необхідності переписування та рефакторингу логіки самого C#-скрипту[6];

2. подієва модель життєвого циклу: використання вбудованих повідомлень двигуна дозволяє ефективно розподіляти обчислювальні потужності. Обробка входу гравця в зону скримеру інвалідного візка не навантажує центральний процесор циклічними перевірками в Update, а викликається дискретно і строго в момент фізичного перетину колізійних меж через метод OnTriggerEnter[4].

3.2.2 Загальна структура ігрового проєкту

Архітектурна організація файлової структури проєкту та ієрархії сцени підпорядкована суворому модульному розподілу, що забезпечує швидкий доступ до ресурсів та мінімізує ризики виникнення конфліктів асетів при повторних збірках. Проєкт розділено на логічні директорії: _Scripts, _Prefabs, _Audio, _Scenes.

Взаємозв'язок основних керуючих компонентів, елементів головного меню та логіки ініціалізації середовища відображено на діаграмі класів ядра та менеджменту аномалій (див. рис. 3.2)[11].

Для деталізації внутрішньоігрової взаємодії розроблено окремі структурні модулі. Так, архітектурна побудова та логіка поведінки персонажа користувача представлені на діаграмі класів контролера гравця (див. рис. 3.3). У свою чергу, статичні та динамічні об'єкти оточення, з якими можлива взаємодія, винесені на діаграму класів інтерактивних об'єктів (див. рис. 3.4).

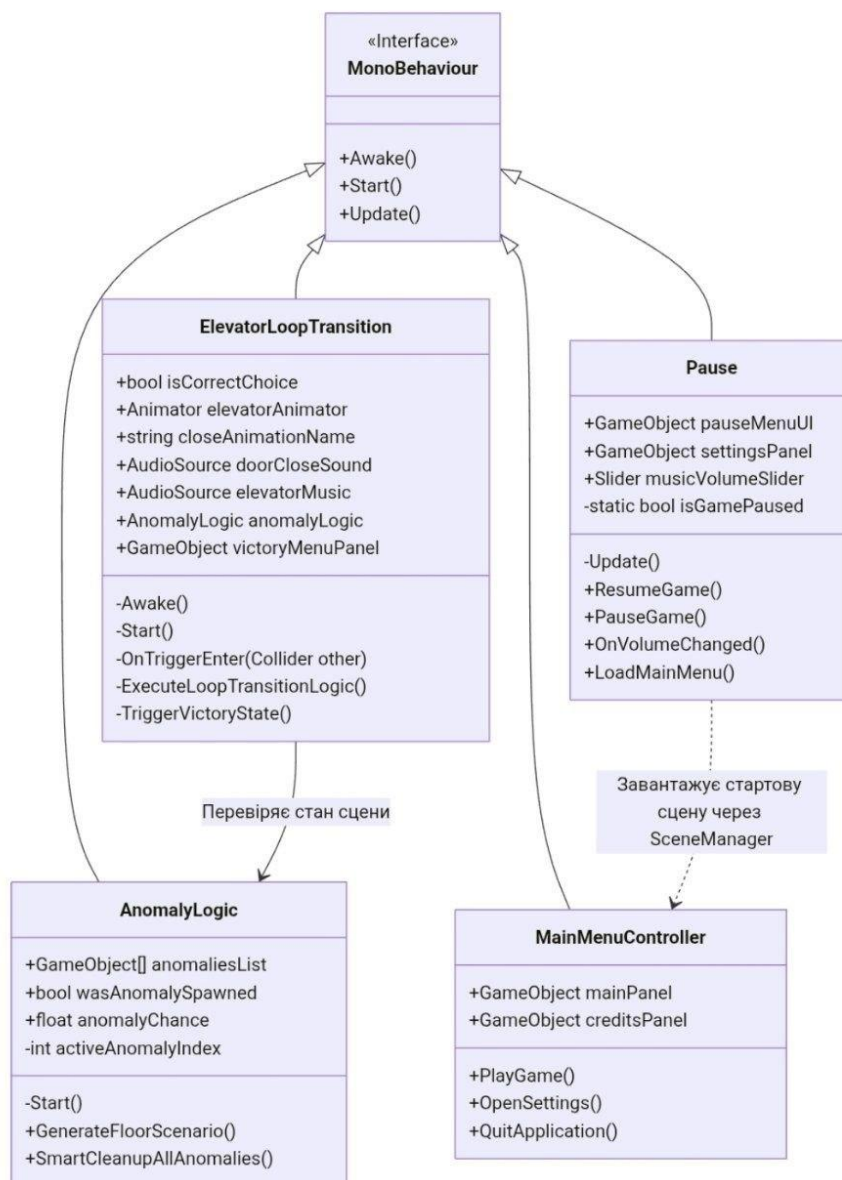


Рис. 3.2 – Діаграма класів глобальні менеджери та інтерфейс користувача

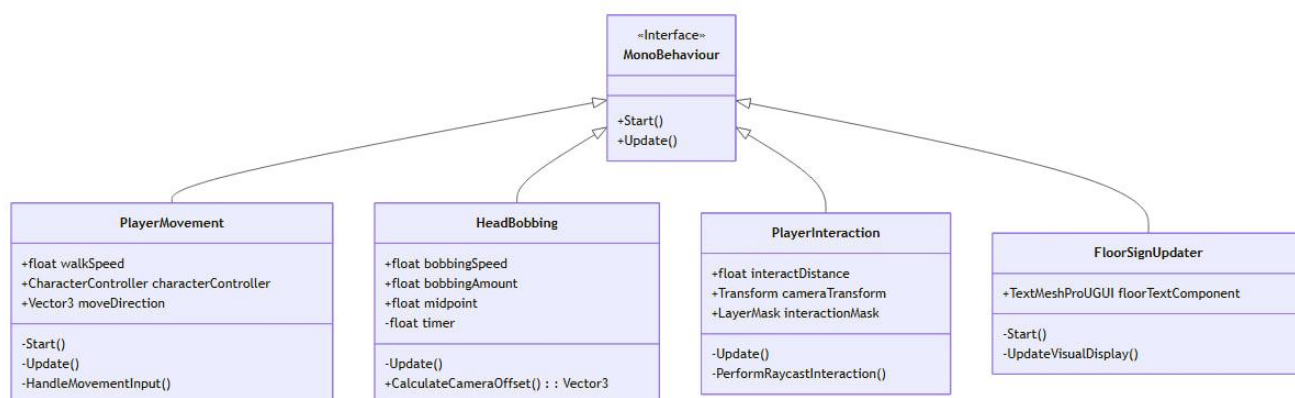


Рис. 3.3 Діаграма класів система гравця та ядра гри

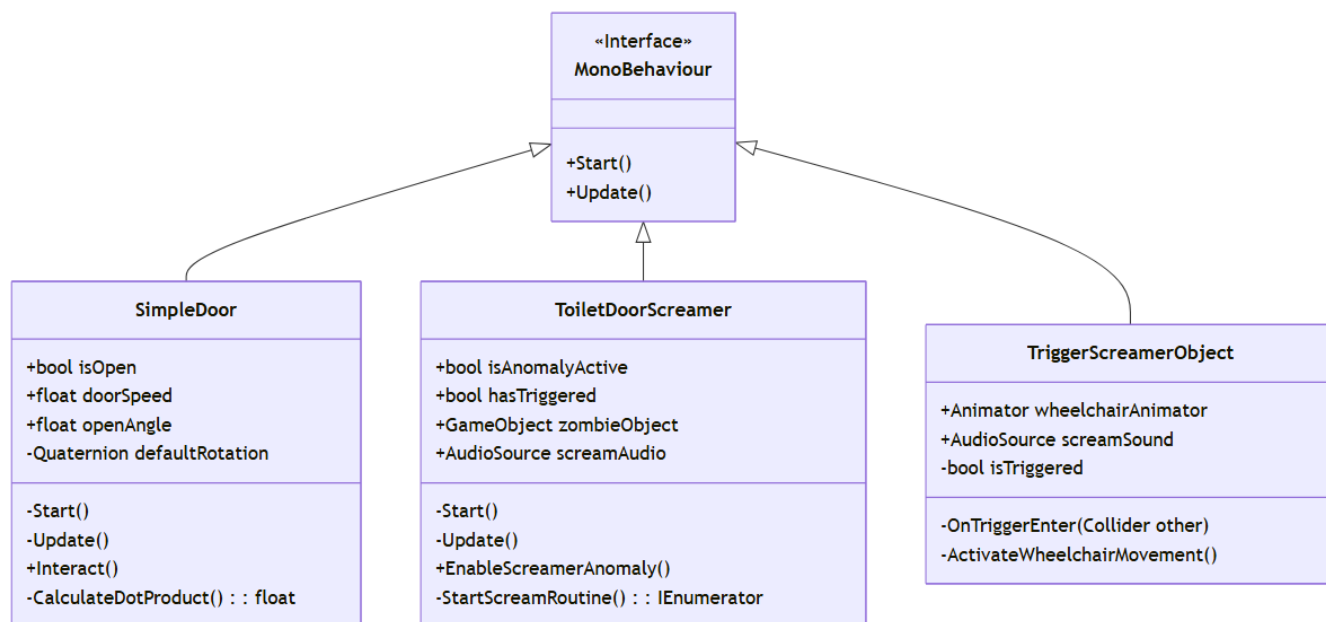


Рис. 3.4 Діаграма класів інтерактивні об'єкти та аномалії

3.2.3 Архітектурна організація менеджерів стану

Центральне керування логічними процесами в застосунку покладено на ієрархічну систему менеджерів станів. Замість збереження всіх змінних в одному глобальному об'єкті, проєкт використовує розподіл зон відповідальності[10]:

менеджер ітерацій (ElevatorLoopTransition): відповідає за глобальний стан прогресу користувача. Він оперує метаданими, що зберігаються в PlayerPrefs[5]. Його завдання — зафіксувати момент, коли гравець залишає поточний поверх, порівняти прапорець наявності аномалії з вектором руху гравця і прийняти рішення: або викликати PlayerPrefs.SetInt("CurrentLoop", currentLoop + 1) та виконати інкремент, або скинути значення на 0. Після цього менеджер ініціює перезавантаження сцени[5];

менеджер ініціалізації аномалій (AnomalyLogic): керує станом поточної локації безпосередньо після її завантаження[1]. Він виконує роль фабрики аномальних об'єктів. Скрипт містить посилання на всі потенційні точки деформації простору. Шляхом розрахунку псевдовипадкового числа він визначає сценарій для поточного поверху. Якщо випадає «чистий поверх», менеджер блокує запуск будь-яких скримерів, переводячи систему в стан спокою[11].

3.2.4 Діаграма послідовності для генерації етажу

Процес послідовної взаємодії між сутностями під час ініціалізації нового поверху має суворий часовий розподіл. Важливо забезпечити, щоб об'єкти-аномалії дізнавалися про свій стан до того, як гравець отримає можливість вийти з ліфта і почати огляд коридору(див. рис. 3.5)[7].

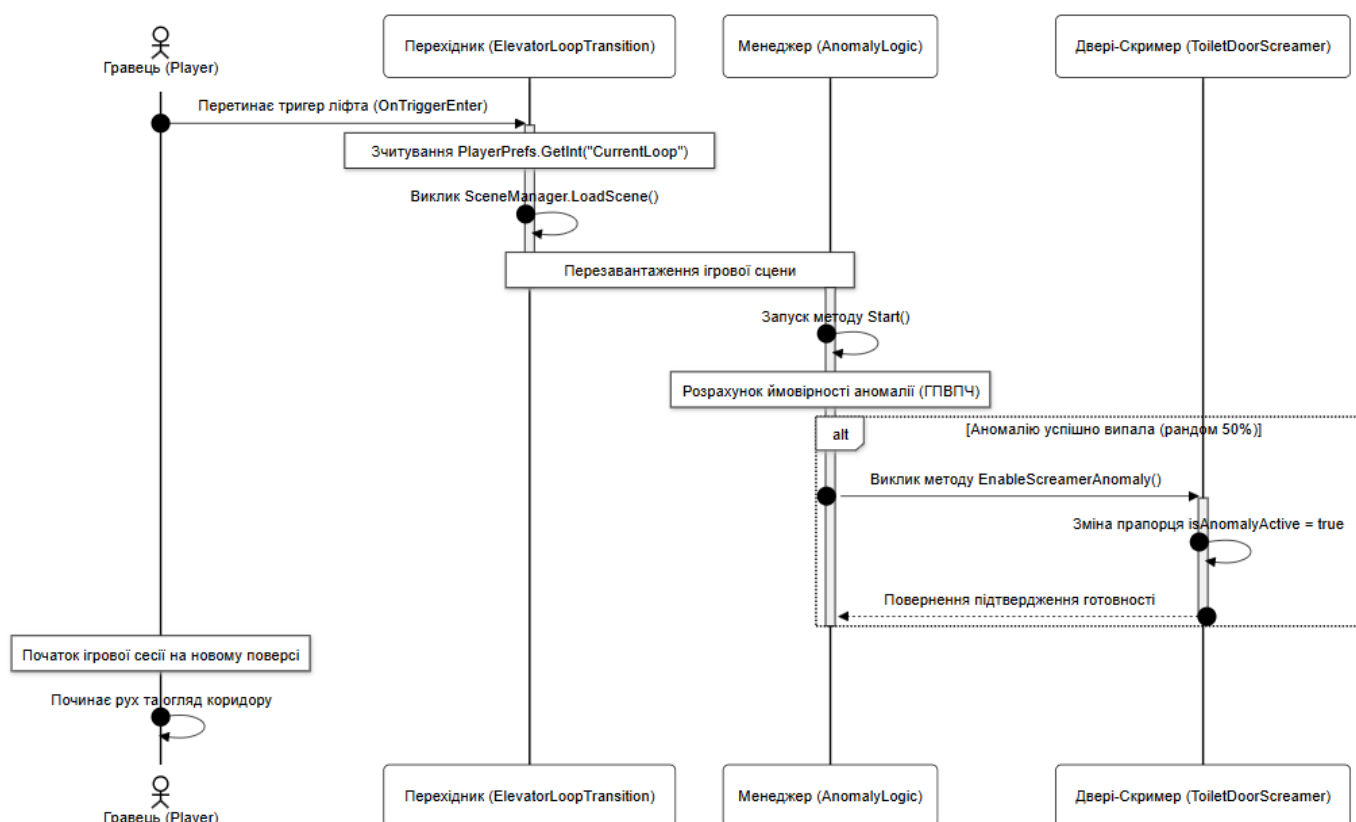


Рис 3.5 Діаграма послідовності завантаження аномалій

3.2.5 Діаграма діяльності системи

Ключовий алгоритм геймплею — це розгалужений процес прийняття рішень, який спрацьовує, коли гравець доходить до кінця коридору. Програма аналізує логічну істинність дій користувача(див. рис. 3.6)[7].

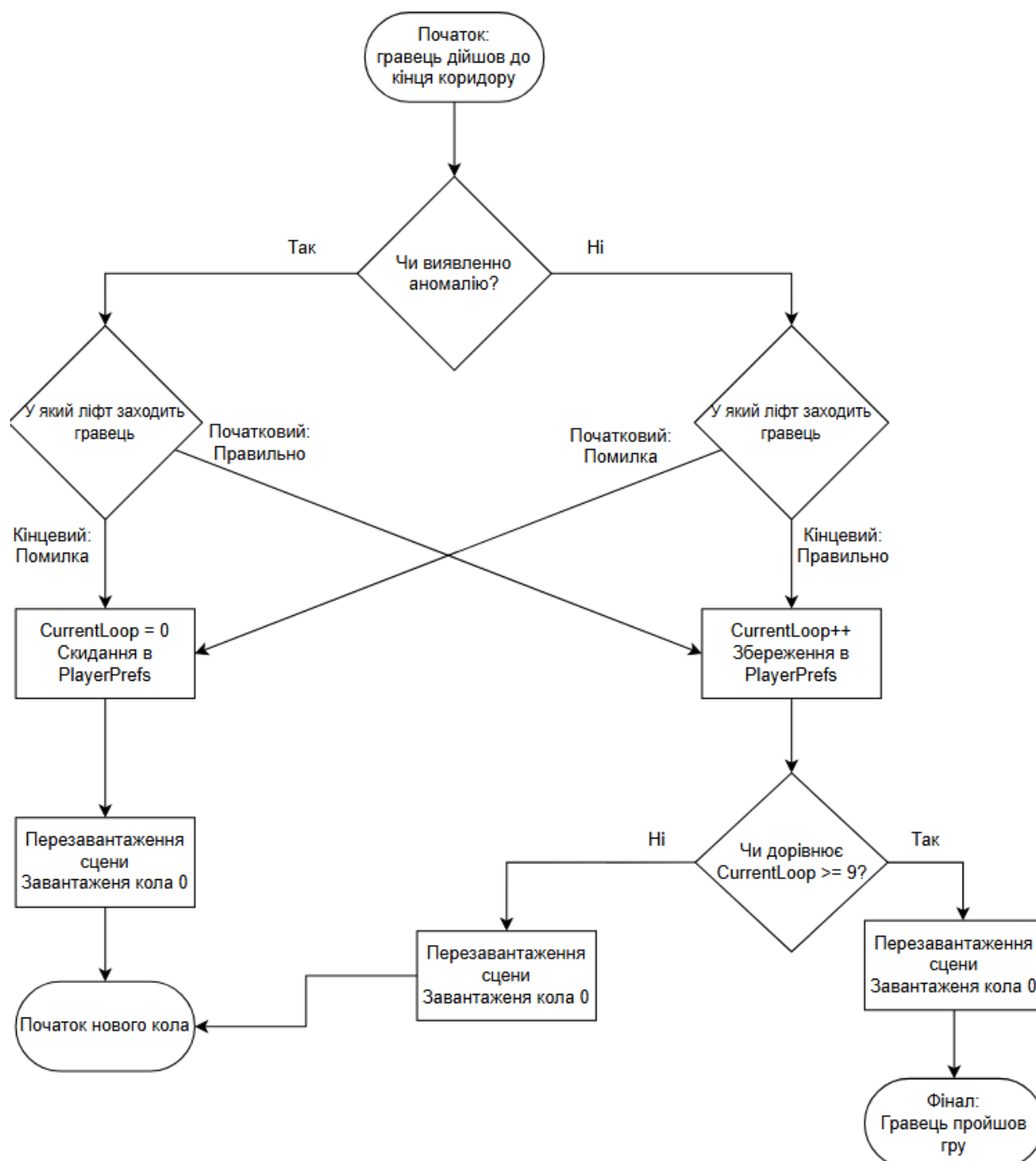


Рис 3.6 Діаграма діяльності алгоритму перевірки ігрових циклів

3.3 Алгоритм роботи розумного очищення та активації аномалій!!!

Для забезпечення стабільної роботи гри без критичних багів було реалізовано алгоритм Smart Cleanup[11]. Його математична та логічна суть полягає в тому, що система перед прийняттям рішення про активацію будь-якої модифікації простору примусово скидає всі об'єкти сцени до базового еталонного стану[1].

Псевдокод алгоритму, що інтегрований у AnomalyLogic[6]: отримати масив усіх зареєстрованих об'єктів-аномалій $A = \{a_1, a_2, \dots, a_n\}$.

Для кожного елемента $a_i \in A$ виконати: знеструмити та деактивувати динамічний меш: $a_i.SetActive(false)$;

Якщо об'єкт містить компонент скримера, примусово скинути логічні прапорці: $hasTriggered = false$, $isAnomalyActive = false$.

Викликати генератор випадкових чисел $R \in [0, 100]$.

Якщо $R \leq AnomalyChance$: вибрати випадковий індекс аномалії $A_{in} = Random.Range(0, A.Count)$;

Активувати обраний об'єкт: $A[A_{in}].SetActive(true)$;

Якщо обрана аномалія — це двері туалету, викликати метод доступу: $A[A_{in}].GetComponent<ToiletDoorScreamer>().EnableScreamerAnomaly()$.

Цей підхід повністю гарантує, що якщо на 1-му колі інвалідний візок котився і кричав, то при переході на 2-ге чисте коло він буде примусово повернутий у вихідну точку, його анімація зупиниться, а звук буде заблоковано[9].

3.4 Вибір і обґрунтування підходу до локального збереження даних

У межах архітектурного проєктування гри було проведено аналіз методів персистенції даних між ітераціями завантаження сцен. Розглядалися три варіанти: запис у текстові файли JSON/XML, розгортання локальної бази даних SQLite та використання вбудованого підсистемного сховища Unity — PlayerPrefs.

Обрано підхід PlayerPrefs з огляду на такі технічні обґрунтування[5]:

низькі накладні витрати: гра оперує виключно одним типом даних для збереження прогресу — цілочисельною змінною[6]. Розгортання повноцінної реляційної бази даних або постійний парсинг файлів JSON через JsonUtility є надлишковим і вимагає додаткових обчислювальних ресурсів на операції дискового введення-виведення[10];

швидкодія: PlayerPrefs зберігає дані у реєстрі операційної системи[5]. Швидкість зчитування та запису однієї змінної з реєстру відбувається за частки

мілісекунд, що повністю виключає мікро-фризи під час переходу між ліфтовими зонами;

надійність: дані сховища автоматично персистуються при виклику `PlayerPrefs.Save()`, що захищає прогрес гравця від втрати навіть у разі аварійного завершення процесу гри[5].

4 ПРОЕКТУВАННЯ СТРУКТУРИ ПРОГРАМИ

4.1 Обґрунтування моделі розробки ігрового застосунку

Для проектування та створення тривимірного психологічного хоррору було обрано ітеративну модель розробки[10]. Специфіка GameDev-проектів такого жанру вимагає постійного тестування ігрового процесу на ранніх етапах для перевірки психологічного сприйняття атмосфери, таймінгів скрімерів та коректності роботи просторових тригерів[11].

Впровадження ітеративного підходу дозволило розділити розробку на кілька послідовних фаз:

1. створення мінімально життєздатного продукту — базового замкненого коридору та фізики переміщення гравця;
2. поетапне нарощування функціоналу — інтеграція фабрики аномалій та системи локальних збережень;
3. налаштування аудіовізуальних ефектів та фінальне полірування ігрового балансу.

4.2 Середовище розробки та технічний стек

Програмна реалізація тривимірного ігрового застосунку «Endless Hallway: Asylum» здійснювалася на базі сучасного інструментарію, що відповідає промисловим стандартам індустрії розробки інтерактивного програмного забезпечення та тривимірного моделювання:

1. ігровий двигун: Unity 6, що забезпечує оптимальну роботу з багатопотоковістю, компіляцією під цільові платформи та просунутими інструментами профілювання[1];
2. мова програмування: C# з підтримкою стандартів .NET, інтегрована в середовище розробки Microsoft Visual Studio 2022[6];

3. графічний конвеєр: Universal Render Pipeline. Вибір цього конвеєра зумовлений необхідністю гнучкого керування ресурсами графічного процесора, оптимізації рендерингу динамічних тіней та постобробки в умовах обмежених просторів[2];

4. пакет тривимірної графіки (3D-модельовання): Blender. Дане середовище використовувалося для створення, оптимізації та текстурування низькополігональних моделей елементів оточення, інтер'єру психіатричної лікарні, а також для налаштування правильної топології сіток і UV-розгортки перед імпортом в ігровий рушій[8];

5. зовнішні репозиторії ресурсів: Unity Asset Store. Платформа застосовувалася як допоміжне середовище для інтеграції готових звукових ефектів, плагінів для роботи з текстовими компонентами та окремих базових шейдерів постобробки[1];

6. система контролю версій та хостинг коду: Git, хмарний репозиторій GitHub та графічний клієнт GitHub Desktop. Цей комплекс інструментів забезпечив детерміноване резервне копіювання проєкту, фіксацію проміжних станів розробки, відстеження працездатності коду та безпечне злиття архітектурних модулів без ризику втрати даних чи пошкодження структури Unity-сцени.

4.3 Вбудовані підсистеми двигуна

Архітектура застосунку спирається на три базові підсистеми ігрового двигуна Unity, які дозволяють досягти високого рівня занурення без перевитрати системних ресурсів:

1. Lighting & Post-Processing (Освітлення та постобробка): використано можливості конвеєра URP для створення гніючої атмосфери замкненого простору лікарні[2]. Завдяки налаштуванню профілів постобробки було інтегровано ефекти затінення Ambient Occlusion, зернистості та корекції кольору, що маскують низькополігональність моделей та підкреслюють хоррор-стилістику;

2. 3D Audio (Просторова аудіосистема): базується на розрахунку відстані та кута між джерелом звуку та приймачем на камері гравця[3]. Це дозволяє реалізувати

дискретне позиціонування звуків скрімерів, кроків та ембієнту, створюючи у гравця відчуття реального об'ємного звуку;

3. Physics Raycasting (Проектування фізичних променів): забезпечує механіку взаємодії гравця з оточенням та фіксацію погляду. З камери гравця методом Physics.Raycast випускається невидимий вектор обмеженої довжини, що визначає колізії з інтерактивними об'єктами на певному прошарку[4].

З точки зору фізичної архітектури, всі вищеописані підсистеми ігрового двигуна та користувацькі налаштування після проходження конвеєра компіляції розгортаються у вигляді набору взаємопов'язаних модулів. На рисунку 4.3 представлено діаграму компонентів, що відображає структуру виконуваного файлу, підключених динамічних бібліотек двигуна Unity та скомпільованих збірок користувацьких C#-скриптів(див. рис. 4.3)[7].

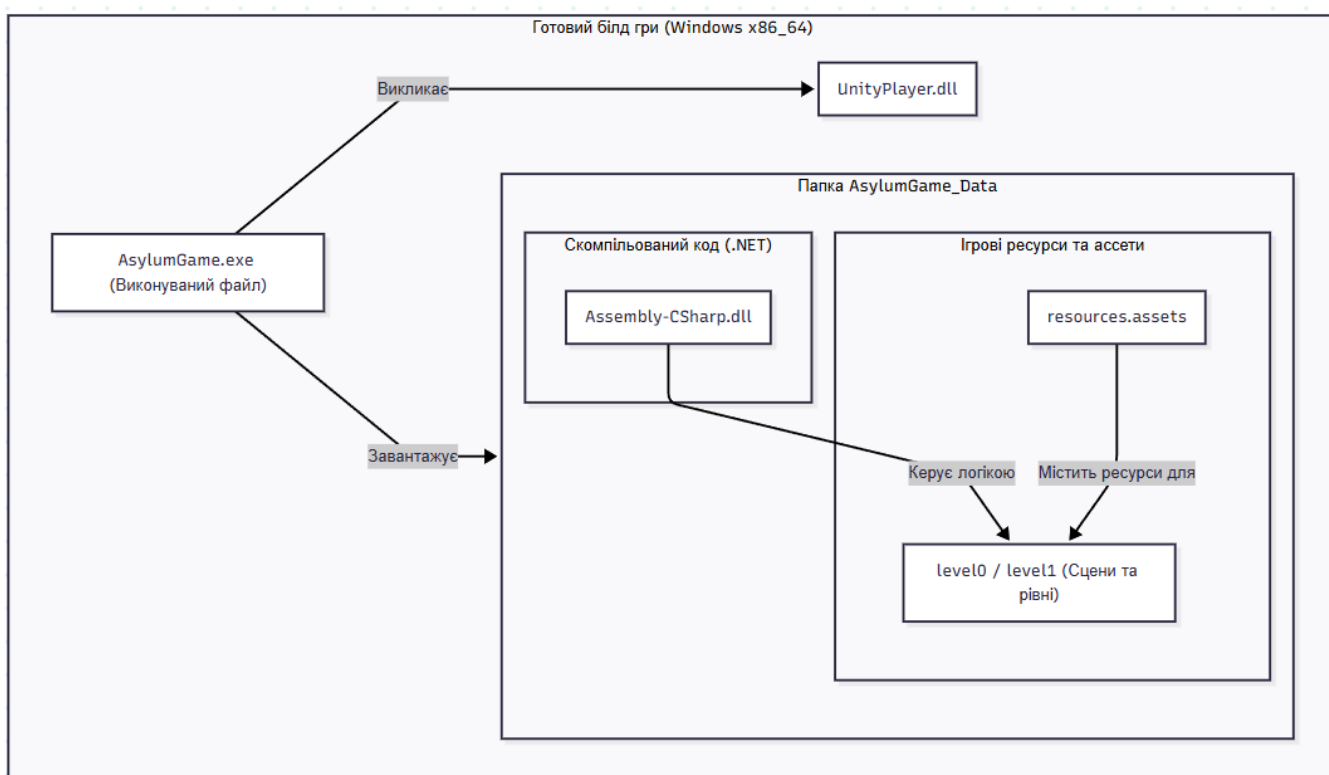


Рис. 4.3 Діаграма компонентів скомпільованого білду ігрового додатку

4.4 Побудова програмного рішення

4.4.1 Логіка випадкової підміни об'єктів та умов перемоги

Центральним вузлом ігрової логіки, що керує ітераціями циклічного простору та валідацією дій гравця, є модуль ElevatorLoopTransition[1]. Архітектурно цей компонент прив'язаний до фізичної тригерної зони ліфта[4]. Взаємодія з ігровим світом побудована на динамічному зчитуванні стану сцени та керуванні анімаційними контролерами дверей.

Замість надлишкового лістингу всього класу, ключову проєктну цінність представляє магістральний метод обробки фізичних колізій та перевірки умов вибору гравця ExecuteLoopTransitionLogic[9]. Нижче наведено фрагмент коду, що відображає математичну та логічну структуру перевірки проходження рівнів та виклику стану абсолютної перемоги[6]:

```
private void ExecuteLoopTransitionLogic()
{
    if (elevatorAnimator != null) elevatorAnimator.Play(closeAnimationName);
    if (doorCloseSound != null) doorCloseSound.Play();
    if (elevatorMusic != null) elevatorMusic.Stop();

    int currentLoop = PlayerPrefs.GetInt("CurrentLoop", 0);
    bool anomalyPresent = anomalyLogic != null &&
anomalyLogic.wasAnomalySpawned;

    if ((anomalyPresent && !isCorrectChoice) || (!anomalyPresent &&
isCorrectChoice))
    {
        currentLoop++;
        PlayerPrefs.SetInt("CurrentLoop", currentLoop);
        PlayerPrefs.Save();

        if (currentLoop >= 9)
        {
            TriggerVictoryState();
            return;
        }
    }
    else
    {
        PlayerPrefs.SetInt("CurrentLoop", 0);
        PlayerPrefs.Save();
    }
}
```

```

    }
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
}

```

Даний метод демонструє застосування паттерна умовного розгалуження на основі персистентних даних[5]. Використання логічної конструкції дозволяє ефективно обробляти два коректних сценарії: коли гравець розвернувся за наявності аномалії та коли гравець пройшов уперед за її відсутності[11]. Для розгортання стану перемоги метод звертається до підсистеми `TriggerVictoryState()`, яка активує панель `VictoryMenuPanel` та повністю зупиняє розрахунки фізичного часу[1].

4.4.2 Програмування контролера та 3D-аудіо у вікнах

Інтерфейс користувача інтегрований із загальною державною машиною застосунку[10]. Для забезпечення коректного перемикання контекстів введення між керуванням персонажем від першої особи та взаємодією з елементами UI розроблено клас `Pause`[1].

Для чіткої інкапсуляції станів ігрового процесу та механізму керування часовим масштабом, у проекті розгорнуто кінцевий автомат. На рисунку 4.4.2 представлено діаграму станів, що демонструє переходи між головним меню, активним геймплеєм, заморошкою потоку обчислень у меню паузи та фінальною зупинкою системи при досягненні умови перемоги(див. рис. 4.4)[7].

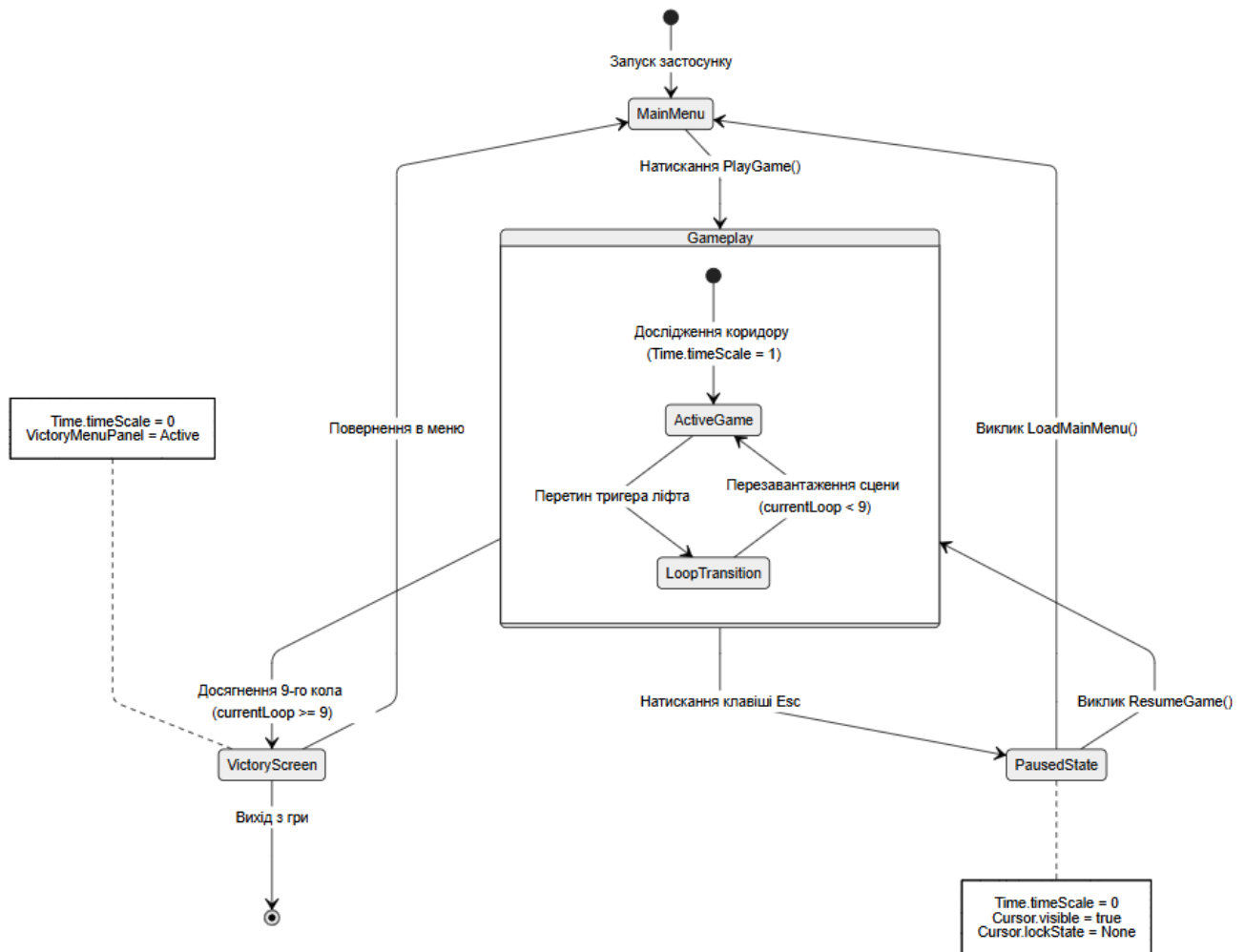


Рис. 4.4 Діаграма станів схема алгоритму перевірки умов проходження коридору

Найбільший інтерес з погляду системного програмування становлять методи інкапсуляції ігрового часу `PauseGame` та `ResumeGame`[6]. Зміна глобального масштабу часу дозволяє миттєво призупиняти виклики системних методів `Update` та розрахунки фізичного рушія `PhysX`[1]:

```

public void ResumeGame()
{
    if (pauseMenuUI != null) pauseMenuUI.SetActive(false);
    if (settingsPanel != null) settingsPanel.SetActive(false);
    Time.timeScale = 1f;
    isGamePaused = false;
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
}

```

```
public void PauseGame()
{
    if (pauseMenuUI != null) pauseMenuUI.SetActive(true);
    Time.timeScale = 0f;
    isGamePaused = true;
    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}
```

Такий підхід гарантує, що у стані паузи ресурси процесора не витрачаються на обробку ігрової біомеханіки персонажа[11].

Паралельно з цим, для посилення атмосфери лімінального простору в межах ігрового циклу, цей же механізм державної машини взаємодіє з дискретним оновленням аудіо-ефектів у зонах віконних рам. При переході між колами аудіосистема динамічно перемикає кліпи із легкого фонового ембієнту на низькочастотні звуки потойбічного вітру, локалізовані біля вікон, змінюючи параметр Spatial Blend на значення 1.0f для повного 3D-позиціонування[3]. Розрахунок загасання звуку відбувається за логарифмічною кривою двигуна Unity, що створює реалістичну акустичну картину без падіння загальної продуктивності та показника кадрів за секунду[3].

5 ТЕСТУВАННЯ

5.1 Що таке тестування в GameDev і навіщо воно потрібне?

Тестування у сфері розробки ігрових застосунків є критично важливим етапом інженерного циклу, спрямованим на верифікацію працездатності програмного коду, виявлення логічних помилок, перевірку фізичних колізій та оцінку стабільності частоти кадрів[10]. Основна мета тестування полягає в забезпеченні відповідності фінального продукту сформованим функціональним та нефункціональним вимогам до моменту релізу[11].

У сучасній практиці проектування інтерактивного програмного забезпечення виділяють два магістральних підходи: автоматизоване та ручне тестування[1]. Автоматизоване тестування використовується для ізольованої перевірки математичних розрахунків та чистого коду[6]. Проте для розробки ігор у жанрі психологічного інді-хоррору ручний метод тестування є пріоритетним з кількох інженерних причин:

1. оцінка атмосфери та таймінгів: жоден автоматичний алгоритм не здатний валідувати психологічний вплив лімінальних просторів, коректність налаштування звукового об'єму та таймінги активації лякаючих елементів[11];

2. емуляція поведінки реального гравця: ручне тестування дозволяє виявити непередбачувані траєкторії руху персонажа, застрягання у геометрії стін та випадкові баги при хаотичному натисканні клавіш керування[9];

3. ефективність при індивідуальній розробці: для фокусних інді-проектів написання складних скриптів автоматизації є надлишковим та економічно недоцільним, тоді як ручне проходження ітерацій дозволяє миттєво фіксувати помилки в консолі редактора Unity[10].

4. у межах даного проекту було проведено серію ручних функціональних тестів для перевірки ключових архітектурних вузлів ігрової системи.

5.2 Тестування інтерфейсу користувача та навігації меню

5.2.1 Головне меню та налаштування аудіопараметрів

Цей етап тестування призначений для перевірки працездатності графічного інтерфейсу користувача у стартовій сцені гри[1]. Основна увага приділяється валідації переходів між вікнами, відгуку кнопок на дії миші та коректності зв'язку повзунків з параметрами гучності(див. рис. 5.2).

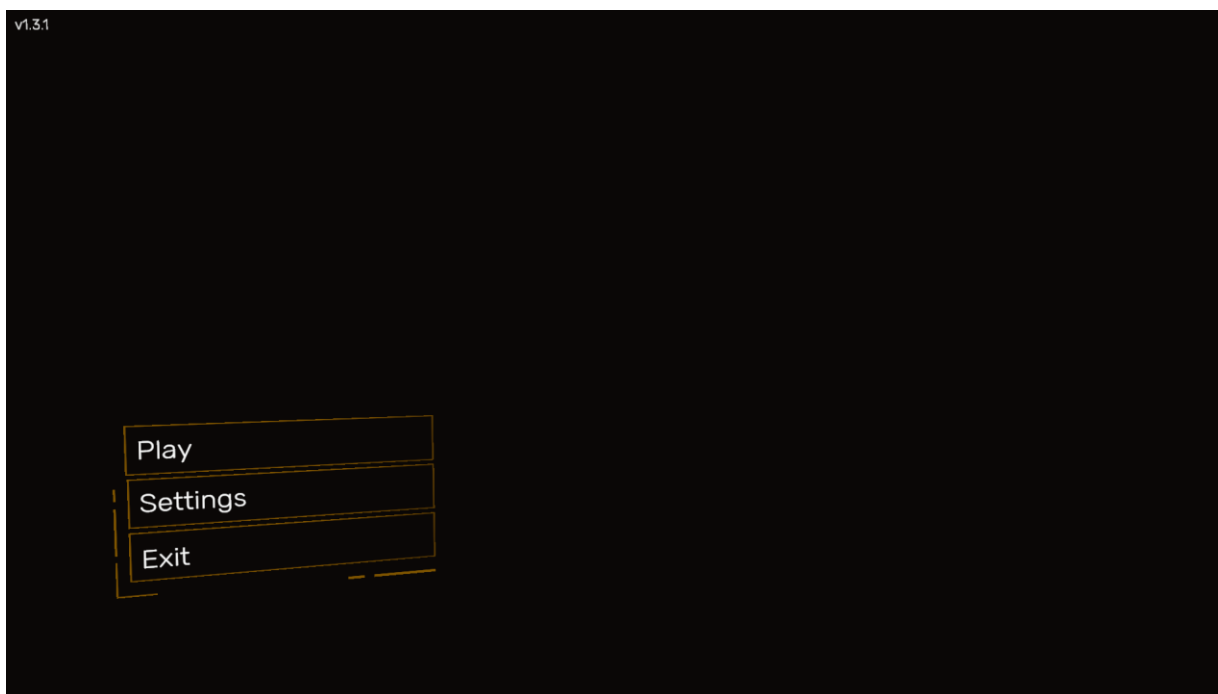


Рис. 5.2 Головне меню

На рисунку 5.2 представлено головне меню гри. В ході ручного тестування було перевірено кожен кнопку на відповідність викликаних методів[10]:

кнопка «Play»: успішно викликає менеджер сцен, ініціюючи завантаження геймплейної локації та обнуляючи попередній прогрес гравця(див рис. 5.3)[1];

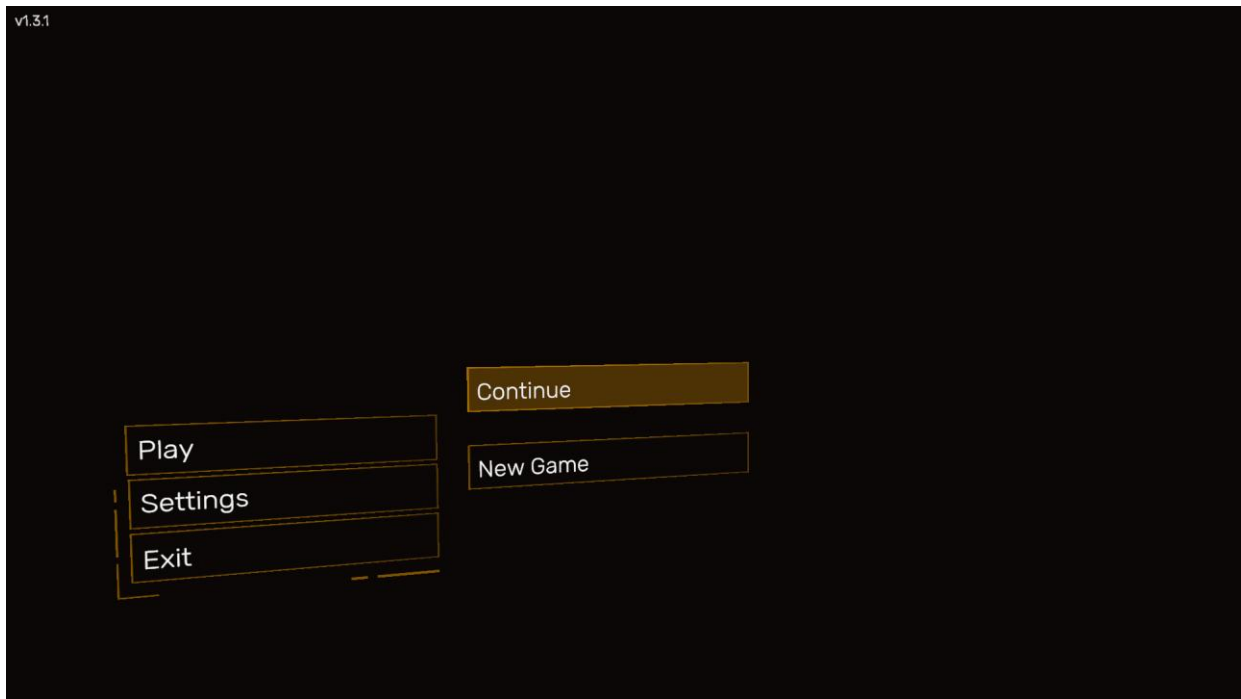


Рис. 5.3 Кнопка грати

Кнопка «Settings»: при натисканні активує дочірній об'єкт панелі налаштувань за допомогою методу SetActive(див. рис. 5.4)[6];

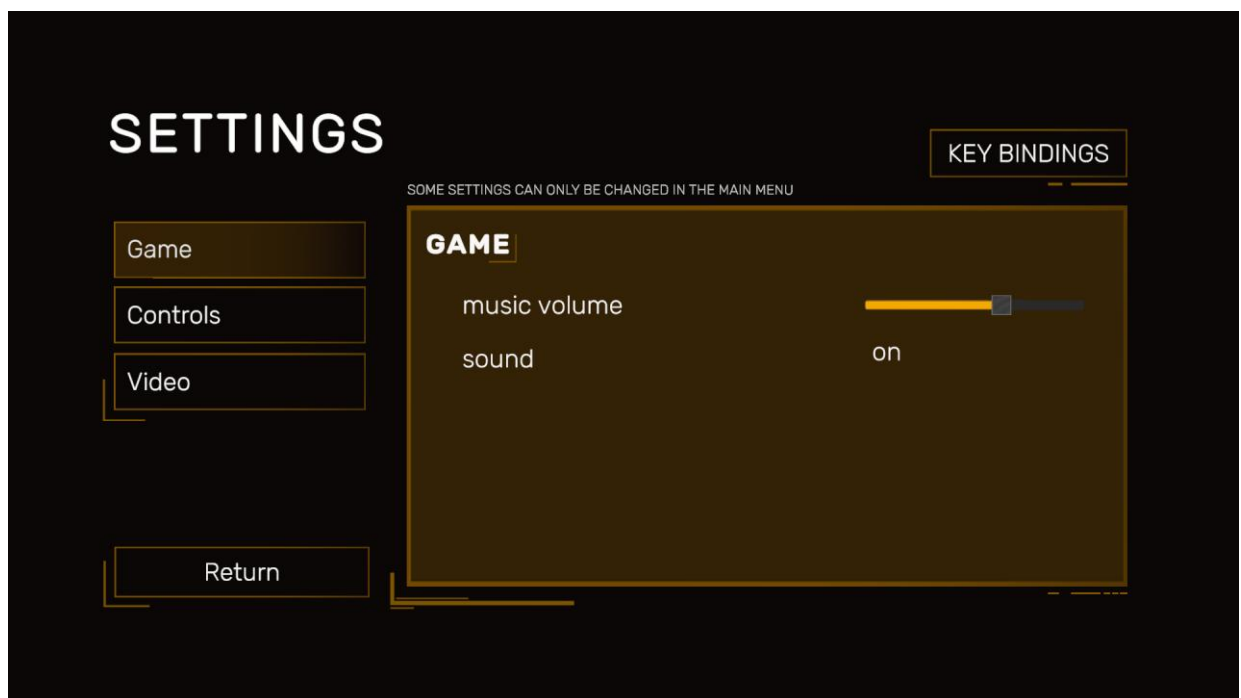


Рис. 5.4 налаштування звуку

Інтерфейс налаштування загальних звукових параметрів представлено на

рисунку 5.4. Панель містить слайдери для динамічного керування аудіомікшером Unity, що дозволяє користувачу окремо регулювати гучність звукових ефектів (SFX) та фонового ембієнту[3].

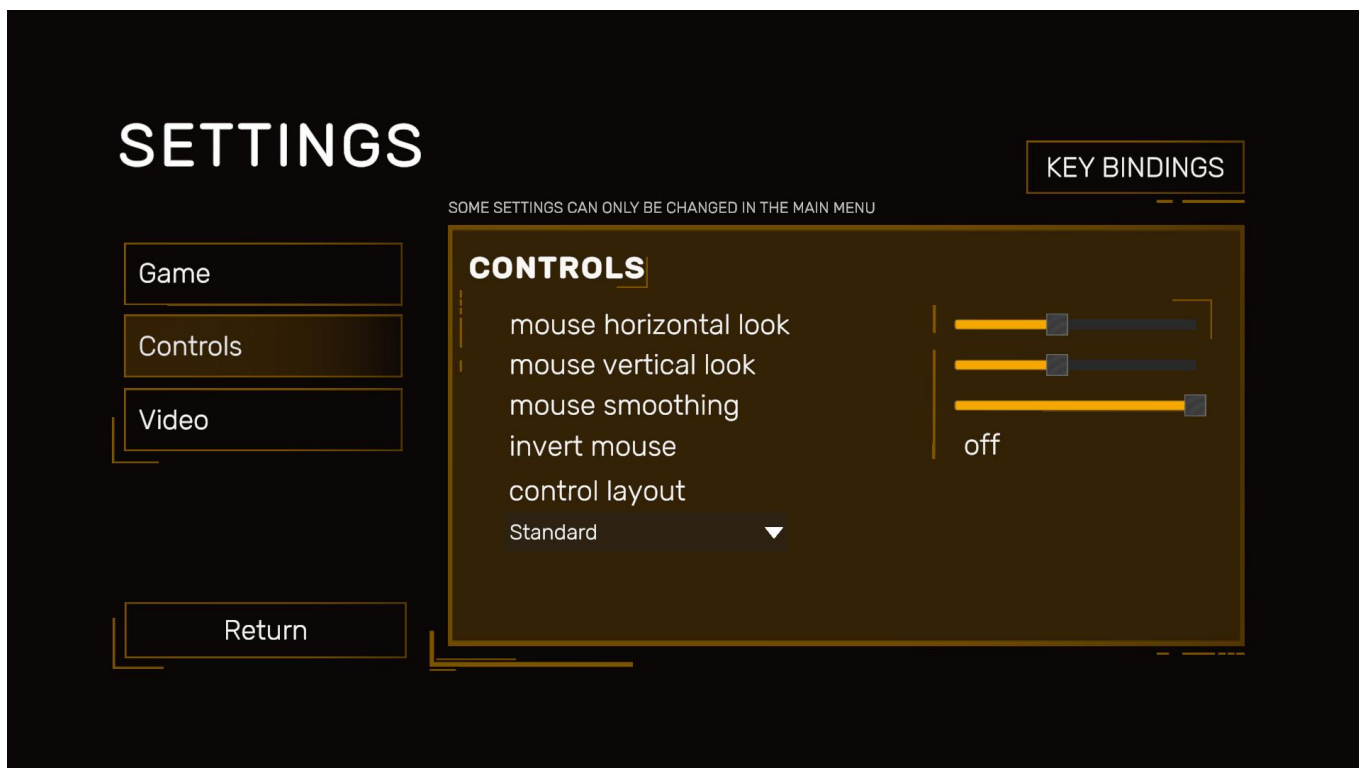


Рис. 5.5 налаштування гри

На рисунку 5.5 продемонстровано інтерфейс налаштування інпут-контролю та чутливості миші. Тут користувач може відкоригувати швидкість обертання камери та інверсію осей, що напряму впливає на обробку системних координат у FPS-контролері персонажа під час гри[9].

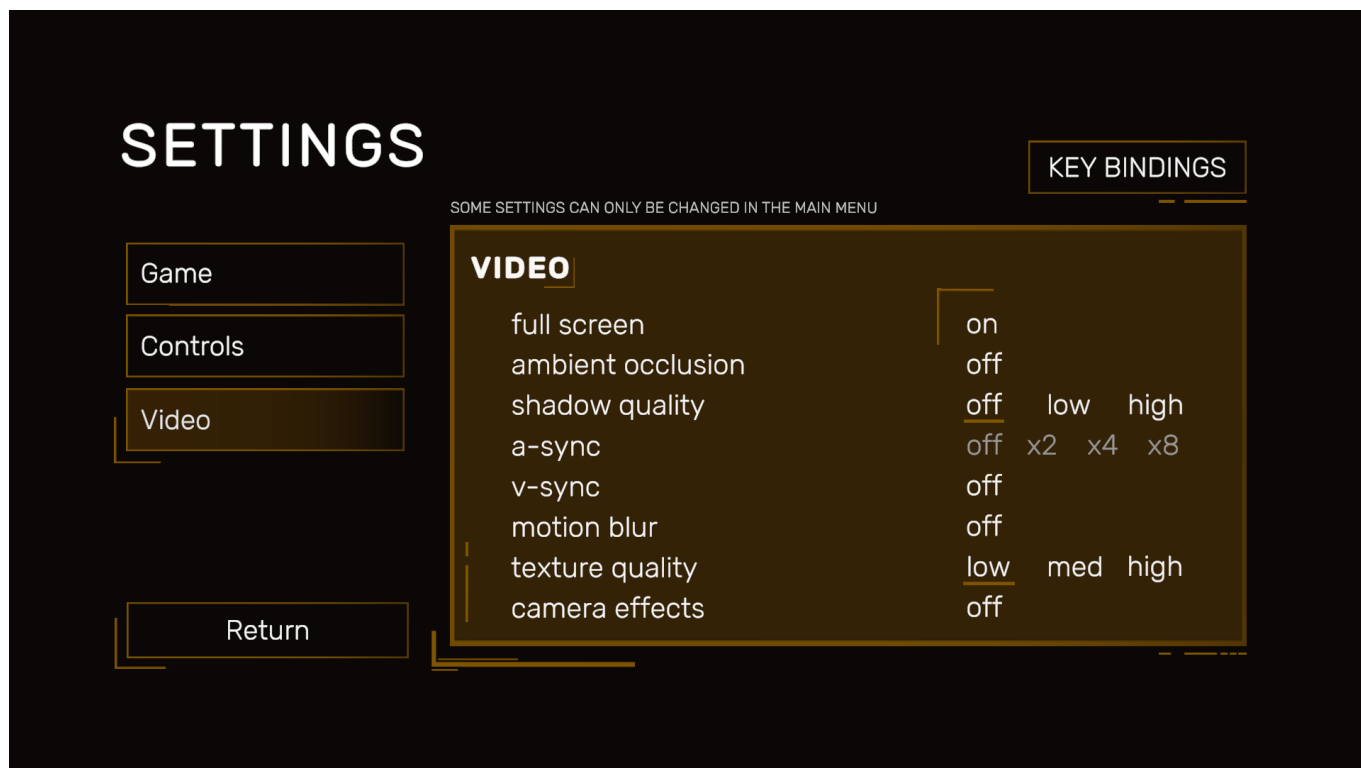


Рис. 5.6 налаштування графіки

На рисунку 5.6 відображено меню конфігурації графічного конвеєра URP (Universal Render Pipeline). Даний екран дозволяє змінювати параметри постобробки, роздільної здатності та згладжування для досягнення оптимального балансу продуктивності й візуальної якості на слабких ПК[2].

Кнопка «Exit»: викликає функцію `Application.Quit()`, яка у скомпільованому білді повністю закриває процес гри(див. рис. 5.7)[1].

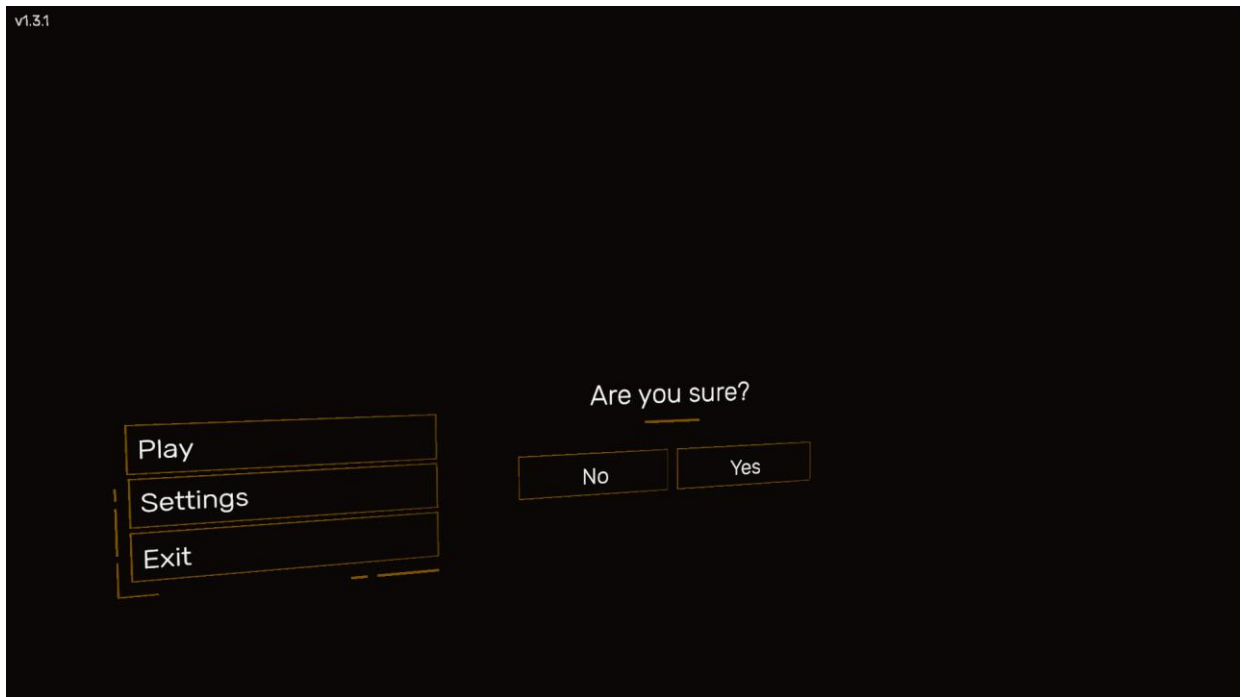


Рис. 5.7 Кнопка «Exit»

5.2.2 Тестування меню паузи в ігровому процесі

Наступним кроком перевіряється робота класу Pause безпосередньо під час ігрового процесу при натисканні клавіші Esc(див. рис. 5.8)[6].

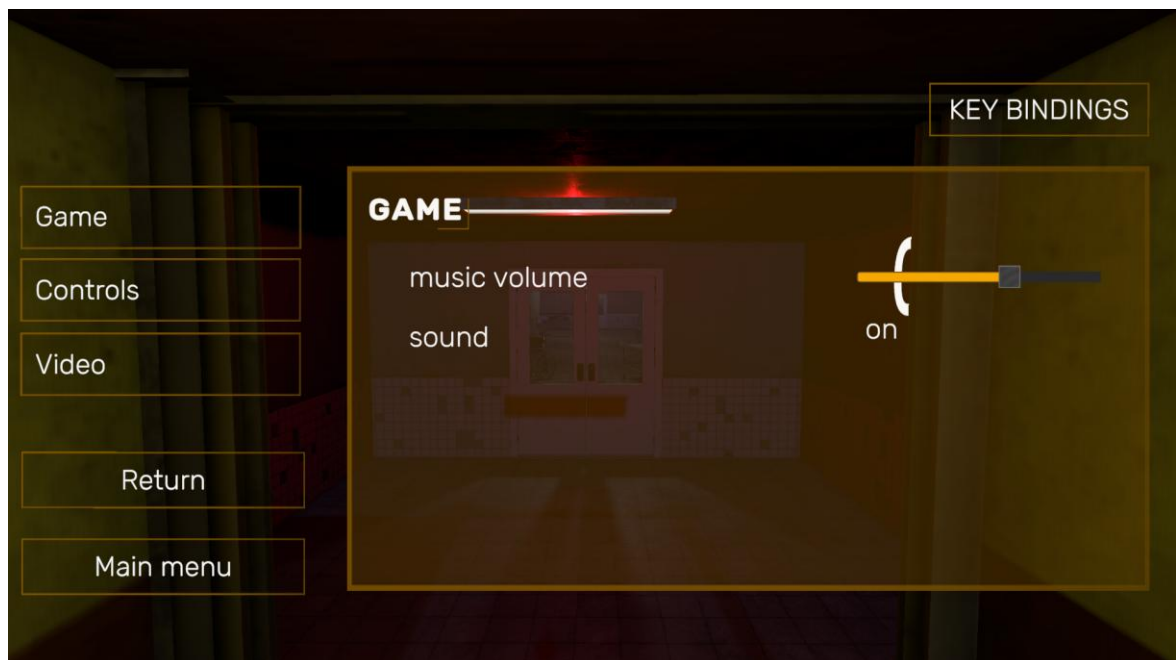


Рис. 5.8 Меню паузи

На рисунку 5.8 зафіксовано момент активації меню паузи. Тестування підтвердило, що при переході у цей стан система виконує три критичні операції[10]:

1. встановлює `Time.timeScale = 0f`, повністю зупиняючи фізику та обчислення в методах `Update`[1];
2. змінює стан курсора на `CursorLockMode.None` та робить його видимим для взаємодії з кнопками меню[6];
3. при повторному натисканні клавіші або кнопки «Продовжити» — повертає гру в активну фазу, ховаючи курсор та відновлюючи хід часу.

Цей етап тестування спрямований на перевірку коректності запису та зчитування персистентних даних за допомогою системного класу `PlayerPrefs` за ключем `"CurrentLoop"`[5]. Алгоритм перевірки передбачає симуляцію двох альтернативних сценаріїв поведінки користувача: правильного вибору напрямку руху та помилкового кроку[11]. Для початку тестування здійснюється запуск сцени з головного меню застосунку(див. рис. 5.9)[1].



Рис. 5.9 Старт гри

Як показано на рисунку 5.9, при першому запуску системи значення

збереженого індексу успішно зчитується як нульове. Наступним кроком тестується механізм успішного проходження: за умови відсутності візуальних змін у коридорі гравець проходить уперед до ліфта. Після перетину фізичного тригера сцени перезавантажується, а лічильник циклів збільшується(див. рис. 5.10)[5].



Рис. 5.10 Валідація успішного запису прогресу в PlayerPrefs

На рисунку 5.10 продемонстровано результат успішного виконання логічного розгалуження в методі `ExecuteLoopTransitionLogic`: система коректно інкрементувала та зберегла дані[6]. Для завершення тесту підсистеми збереження симулюється помилка: за наявності аномалії гравець навмисно ігнорує її та йде вперед. Результатом обробки цієї помилки є повне скидання прогресу — після завантаження сцени табло знову відображає «0», що підтверджує стовідсоткову детермінованість алгоритму валідації[5].

5.3 Тестування відсікання тригерів скримерів на чистому колі

Особливістю архітектури розробленого хоррору є логічне розділення ігрових просторів на «модифіковані» та «чисті»(див. рис. 5.11)[11]. Тестування даного

модуля розробки покликане підтвердити, що у випадку, коли генератор AnomalyLogic видав значення `wasAnomalySpawned = false`, на сцені повністю блокуються та деактивуються всі фізичні тригери[4].

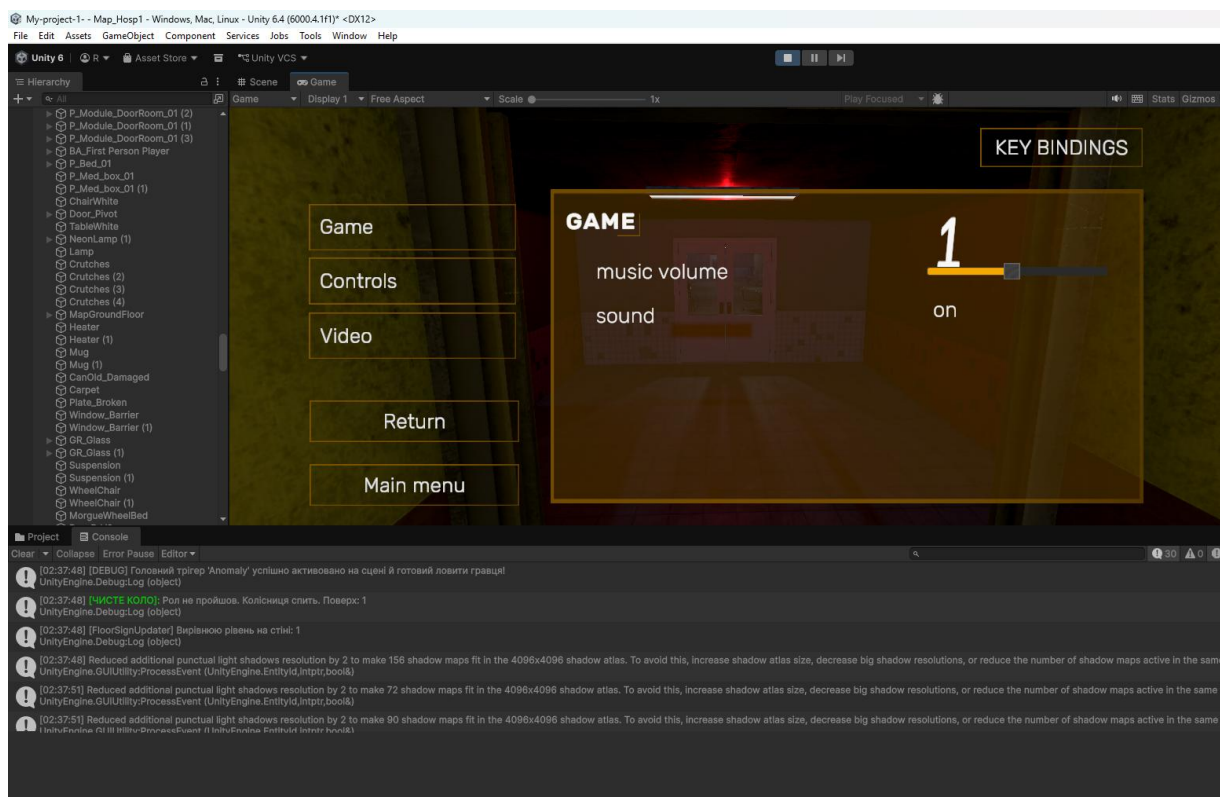


Рис. 5.11 Перевірка стану «чистого кола»

На рисунку 5.11 зафіксовано проходження гравцем коридору у стані відсутності аномалій. Модуль очищення сцени при завантаженні успішно деактивував об'єкти хорор-індустрії, запобігаючи помилковим спрацьовуванням анімацій та звуків[1]. Це гарантує, що гравець не зустрінє скример там, де простір має залишатися абсолютно нормальним, зберігаючи закладені правила психологічного тиску[10].

5.4 Тестування зміни аудіокліпів вікон та блокування паузи при екрані перемоги

Заключний етап тестування присвячений перевірці динамічної модифікації

атмосфери та коректності роботи фінальної державної машини. Згідно з архітектурним планом, при досягненні критичних етапів гри фоновий звук біля вікон має дискретно змінюватися на низькочастотний гул вітру, а при досягненні 9-го кола — викликатися екран перемоги з повним блокуванням системних клавіш (див. Рис. 5.12)[3].



Рис. 5.12 Фінальний стан системи: активація інтерфейсу перемоги

На рисунку 5.12 продемонстровано фінальну фазу функціонального тестування. При досягненні умови `currentLoop >= 9` підсистема `TriggerVictoryState()` миттєво вивела на екран `VictoryMenuPanel[1]`. В процесі тестування було виконано перевірку захисту від збоїв: при активному екрані перемоги натискання клавіші `Esc` повністю ігнорується кодом класу `Pause`, оскільки масштаб часу вже зафіксовано на позначці `Time.timeScale = 0f[6]`.

ВИСНОВКИ

У рамках даної курсової роботи було успішно розроблено ігровий застосунок у жанрі психологічного інді-хоррору — «Endless Hallway: Asylum» — з використанням сучасного кросплатформного двигуна Unity та конвеєра рендерингу URP. Головною метою проекту стало створення функціональної, оптимізованої та атмосферної платформи, призначеної для занурення гравця у лімінальний простір із механікою детективного пошуку аномалій і циклічного переходу між поверхами. Роботу розпочато з комплексного аналізу предметної області, який охопив вивчення історії становлення ігрових механік «симулятора ходіння», аналіз феномену лімінальних просторів у геймдизайні, а також критичний огляд популярних аналогічних проєктів.

На основі проведеного дослідження були чітко сформульовані функціональні, нефункціональні та користувацькі вимоги до майбутньої ігрової системи. Важливим етапом стало проведення аналізу потенційних ризиків, пов'язаних із розробкою тривимірної графіки, генерацією випадкових подій та оптимізацією навантаження на графічний процесор, а також визначення шляхів їхньої мінімізації. Було також чітко окреслено цільову аудиторію проєкту — шанувальників інтерактивних детективних хорорів та атмосферних ігор, що дозволило сфокусувати зусилля на механіках психологічного тиску та аудіовізуального занурення.

Ключовим аспектом роботи стало проєктування архітектури програмного продукту, де перевагу було надано компонентно-орієнтованій моделі рушія Unity, яка забезпечила ефективне розмежування відповідальності між логічними модулями гри, скриптами поведінки персонажа та менеджерами станів. Для наочної візуалізації структури, поведінки та взаємозв'язків у системі було розроблено вичерпний набір діаграм у нотації Mermaid, зокрема фізичну діаграму компонентів білду під Windows x86_64 та скінченний автомат для контролю логіки паузи, ігрового процесу та тригерів перемоги.

Безпосередній процес розробки програмного продукту базувався на ітераційній моделі, що дозволило гнучко підходити до реалізації геймплейних механік та

оперативно реагувати на консольні помилки під час компіляції. Було ретельно підібрано оптимальний технологічний стек, що включає об'єктно-орієнтовану мову програмування C#, платформу Unity для збирання сцени та обробки фізики PhysX, інструментарій компіляції .NET, а також інтегроване середовище розробки Microsoft Visual Studio / Rider та пакет Blender для тривимірного моделювання елементів оточення лікарні.

У ході практичної реалізації було розроблено механізми динамічного переміщення персонажа від першої особи, систему проектування фізичних променів для фіксації погляду гравця, а також інтегровано модуль випадкової генерації візуальних аномалій AnomalyLogic. Значну увагу було приділено як розробці просторової аудіосистеми для посилення ефекту присутності, так і побудові надійної логіки збереження ігрового прогресу через персистентні ключі PlayerPrefs для реалізації механіки 9 кіл пекла ліфта.

Для перевірки коректності роботи основних функціональних блоків системи та відсікання помилкових тригерів скрімерів на «чистих» колах було проведено ручне функціональне тестування. Як підсумок, створений програмний продукт повністю відповідає поставленим на початку роботи вимогам, надаючи користувачам стабільний, оптимізований та дієвий інструмент для інтерактивного проведення дозвілля. Розроблений хоррор-застосунок «Endless Hallway: Asylum» є вдалим прикладом практичного застосування сучасних технологій і підходів до тривимірної розробки та демонструє значний потенціал для подальшого вдосконалення, розширення пулу аномалій та масштабування у повноцінний ігровий реліз.

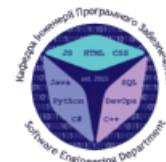
ПЕРЕЛІК ПОСИЛАНЬ

1. Unity - Manual: Programming in Unity. Unity - Manual: Unity 6.4 User Manual.
URL: <https://docs.unity3d.com/Manual/scripting.html> (date of access: 03.06.2026)
2. Universal Render Pipeline overview | Universal RP | 14.0.12. Unity - Manual: Unity 6.4 User Manual. URL: <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@14.0/manual/index.html> (date of access: 03.06.2026).
3. Unity - Manual: Audio Source. Unity - Manual: Unity 6.4 User Manual.
URL: <https://docs.unity3d.com/Manual/class-AudioSource.html> (date of access: 03.06.2026).
4. Unity - Scripting API: Physics.Raycast. Unity - Manual: Unity 6.4 User Manual.
URL: <https://docs.unity3d.com/ScriptReference/Physics.Raycast.html> (date of access: 03.06.2026).
5. Unity - Scripting API: PlayerPrefs. Unity - Manual: Unity 6.4 User Manual.
URL: <https://docs.unity3d.com/ScriptReference/PlayerPrefs.html> (date of access: 03.06.2026).
6. C# Guide - .NET managed language - C#. Microsoft Learn: Build with answers in reach. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/> (date of access: 03.06.2026).
7. About the Unified Modeling Language Specification Version 2.5.1. Object Management Group. URL: <https://www.omg.org/spec/UML/> (date of access: 03.06.2026).
8. Blender 5.1 Manual. Blender Documentation - blender.org.
URL: <https://docs.blender.org/manual/en/latest/> (date of access: 03.06.2026).
9. Thorn A. Pro Unity Game Development with C#. Apress L. P., 2014.
10. Bond J. G. Introduction to Game Design, Prototyping, and Development: From Concept to Playable Game with Unity and C#. Pearson Education, Limited, 2022.
11. Murray J. Building Games with Unity 2021: A Complete Reference for Indie Game Developers. 1st ed. Birmingham : Packt Publishing, 2021. 538 p.

ДОДАТОК А. ДЕМОНСТРАЦІЙНІ МАТЕРІАЛИ



ДЕРЖАВНИЙ УНІВЕРСИТЕТ
ІНФОРМАЦІЙНО - КОМУНІКАЦІЙНИХ ТЕХНОЛОГІЙ
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ІНФОРМАЦІЙНИХ
ТЕХНОЛОГІЙ



Кафедра Інженерії програмного забезпечення

Розробка гри "Endless Hallway: Asylum" у жанрі хоррор з елементами пошуку аномалій

Виконав:

Студент групи ПД-22 Тараканов РодіонСергійович

Керівник:

Ярошевський Олександр Вікторович
асистент кафедри Інженерії програмного забезпечення

Київ - 2026

МЕТА, ОБ'ЄКТА ТА ПРЕДМЕТ ДОСЛІДЖЕННЯ

Мета роботи проектування та практична розробка тривимірного ігрового застосунку «Endless Hallway: Asylum» у жанрі психологічного інді-хоррору з використанням об'єктно-орієнтованого програмування, мови C#, кросплатформного двигуна Unity та конвеєра рендерингу URP [2, 3, 8].

Об'єкт дослідження процес проектування, архітектурної організації, алгоритмічної оптимізації та програмної реалізації тривимірних інтерактивних систем та відеоігор на базі компонентно-орієнтованого підходу

Предмет дослідження алгоритми програмування скінченних автоматів, логіка випадкової генерації подій та архітектура C#-скриптів у середовищі Unity.

АНАЛІЗ АНАЛОГІВ

Параметри порівняння та технічні критерії	The Exit 8 (KOTAKE CREATE)	I'm on Observation Duty	«Endless Hallway: Asylum» (Розроблюваний проєкт)
Базовий інструментарій / Двигун	Unreal Engine 5	Unity 2021/2022	Unity 6 (6000.x.x)
Режим перспективи камери (View)	Від першої особи (First-PersonView)	Дискретні статичні камери (CCTV)	Від першої особи + кастомний Head Bobbing
Умова успішної фіналізації сесії	Досягнення виходу №8	Вживання до 06:00 (таймер)	Досягнення 9-го кола (CurrentLoop >= 9)
Алгоритм валідації вибору шляху	Фізичний розворот на 180°	Надсилення текстової форми	Вибір вектора руху: Вперед або назад
Типи просторових аномалій	Візуальні підміни мешів	Зникнення, спавн, привиди	Комплексні: Swap, Scale, тригерні скримери
Динамічна аудіосистема	Відсутня (статичний ембієнт)	Дискретні шуми перешкод	Прогресивне 3D Audio з симуляцією зливи
Кастомізація введення (Input)	Фіксована (Hardcoded)	Відсутня	Динамічний KeyBindManager (Rebinding)

3

КОНЦЕПТ ГРИ

В основі розробленого ігрового застосунку покладено концепцію дослідження лімінального простору в атмосфері замкненої психіатричної лікарні. Головне завдання гравця — детективний пошук візуальних та аудіальних змін у навколишньому середовищі, покладаючись на власну спостережливість.

Логіка ігрового процесу детермінована й базується на трьох суворих правилах взаємодії з оточенням:

1. Ідентифікація аномалії: якщо під час дослідження коридору гравець виявляє будь-яку структурну, графічну або звукову зміну (аномалію), він повинен негайно розвернутися і піти у протилежному напрямку до початкового ліфта;

2. Еталонний стан (чисте коло): якщо коридор залишається в абсолютно незмінному, базовому стані без жодних деформацій, гравець має продовжувати рух уперед і зайти в ліфт у кінці холу;

3. Умова успішного фіналу: для успішного завершення гри та втечі з аномального простору психіатричної лікарні користувачу необхідно безпомилково розпізнати стани локації та послідовно подолати всі поверхи коридору до досягнення 8 поверху. Коли досягнуто 8 поверх — буде виведена плашка з перемогою.

4

ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Функціональні вимоги:

генерація умов: розрахунок імовірності появи змін у коридорі через `AnomalyLogic` ,
формування масиву `anomaliesList` та очищення сцени методом `SmartCleanupAllAnomalies ()`;
логіка переходів: відстеження тригерів ліфта (`OnTriggerEnter`), перевірка вибору гравця
(`wasAnomalySpawned`) та зміна лічильника кіл;
керування станами: контроль фаз гри через `Pause` та `MainMenuController` , зупинка часу
(`Time.timeScale = 0f`) та активація `VictoryScreen` при досягненні 8 -ого кола .

Нефункціональні вимоги :

продуктивність : стабільні 60+ FPS на Windows x86_64 завдяки оптимізації конвеєра URP та
низькополігональним моделям;

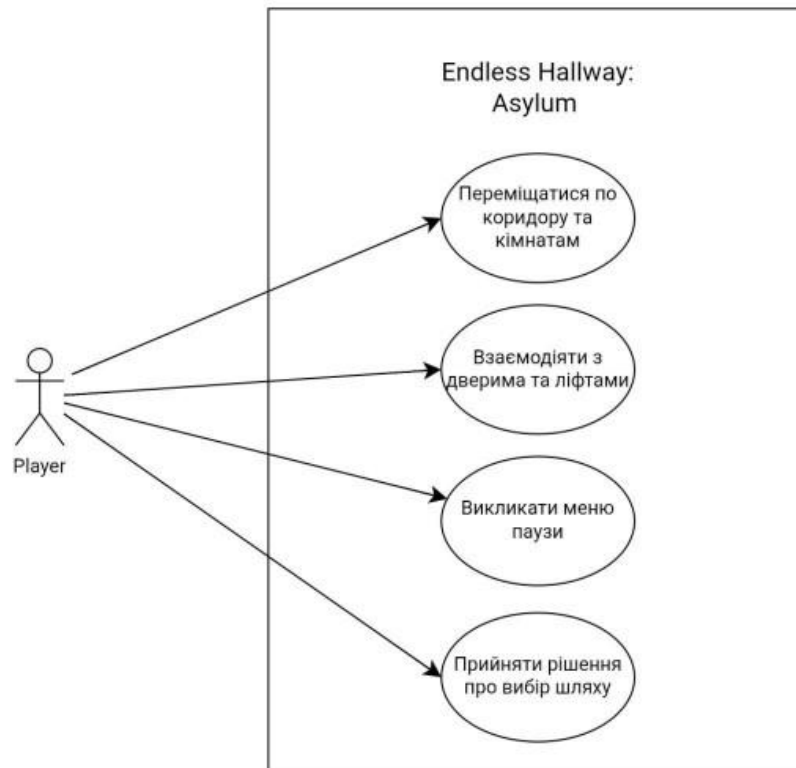
мінімальні системні вимоги (для гри у 720p / 30 FPS): процесор Intel Core i3-4160 / AMD
FX-6300; 8 ГБ оперативної пам'яті; відеокарта Nvidia GeForce GTX 750 Ti / AMD Radeon
R7 260X; 2 ГБ відеопам'яті;

рекомендовані системні вимоги (для гри у 1080p / 60 FPS): процесор Intel Core i5-7400 /
AMD Ryzen 3 1200; 12 ГБ оперативної пам'яті; відеокарта Nvidia GeForce GTX 1050 Ti /
AMD Radeon RX 560; 4 ГБ відеопам'яті;

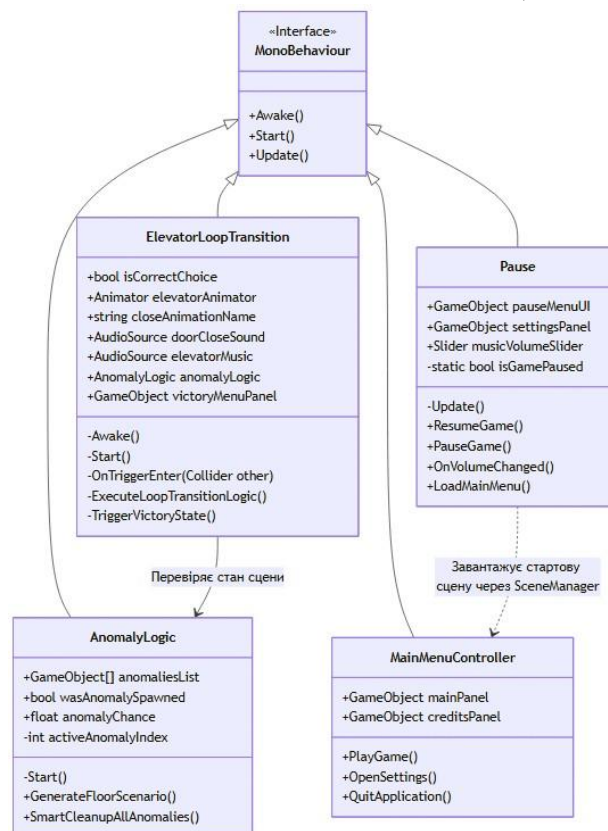
ПРОГРАМНІ ЗАСОБИ РЕАЛІЗАЦІЇ



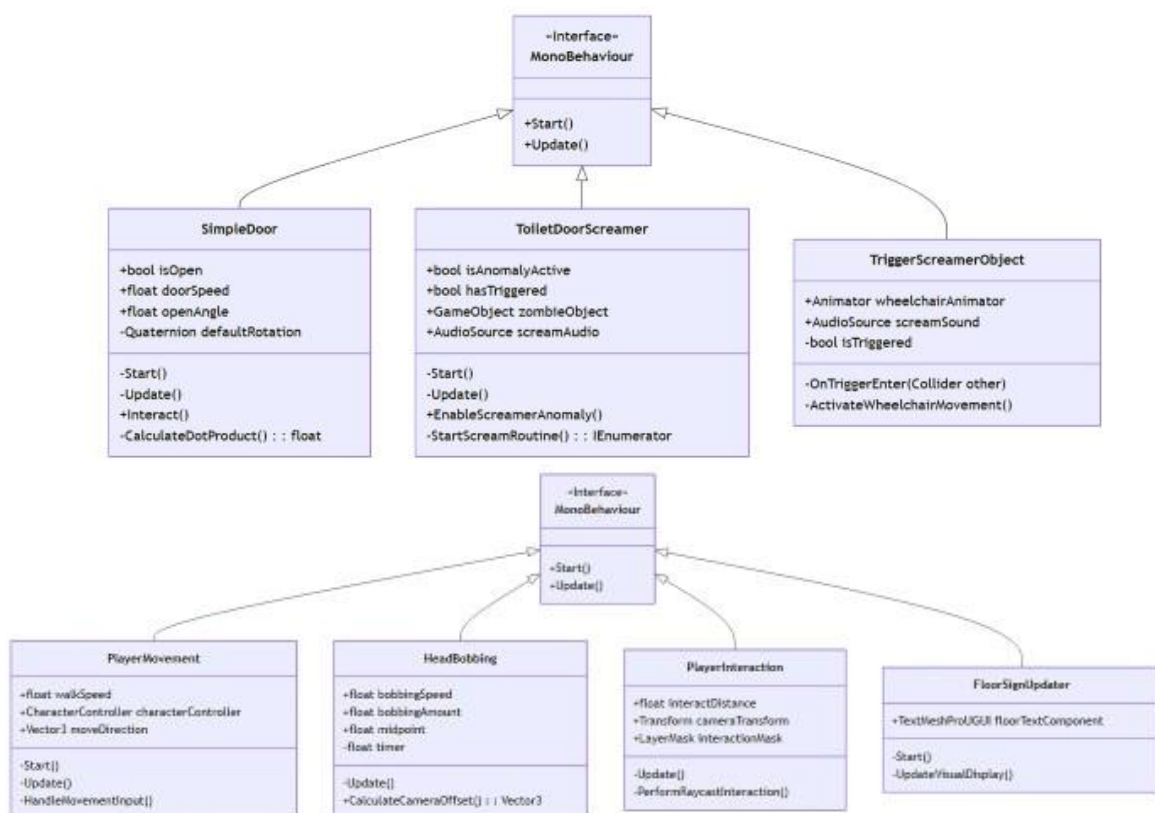
ДІАГРАМА ПРЕЦЕДЕНТІВ ГРАВЦЯ



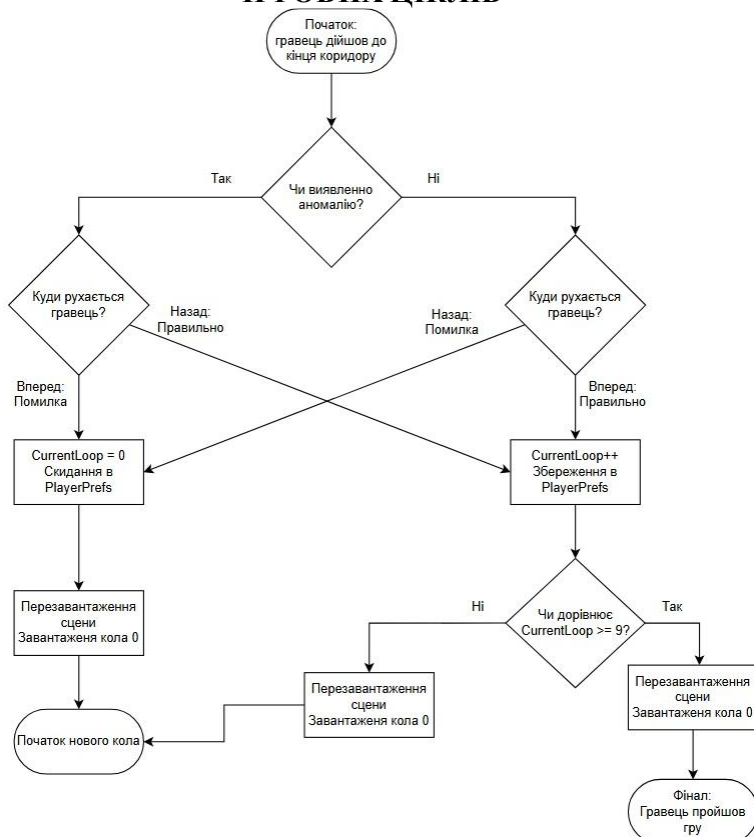
ДІАГРАМА КЛАСІВ(ГЛОБАЛЬНІ МЕНЕДЖЕРИ ТА ІНТЕРФЕЙС КОРИСТУВАЧА)



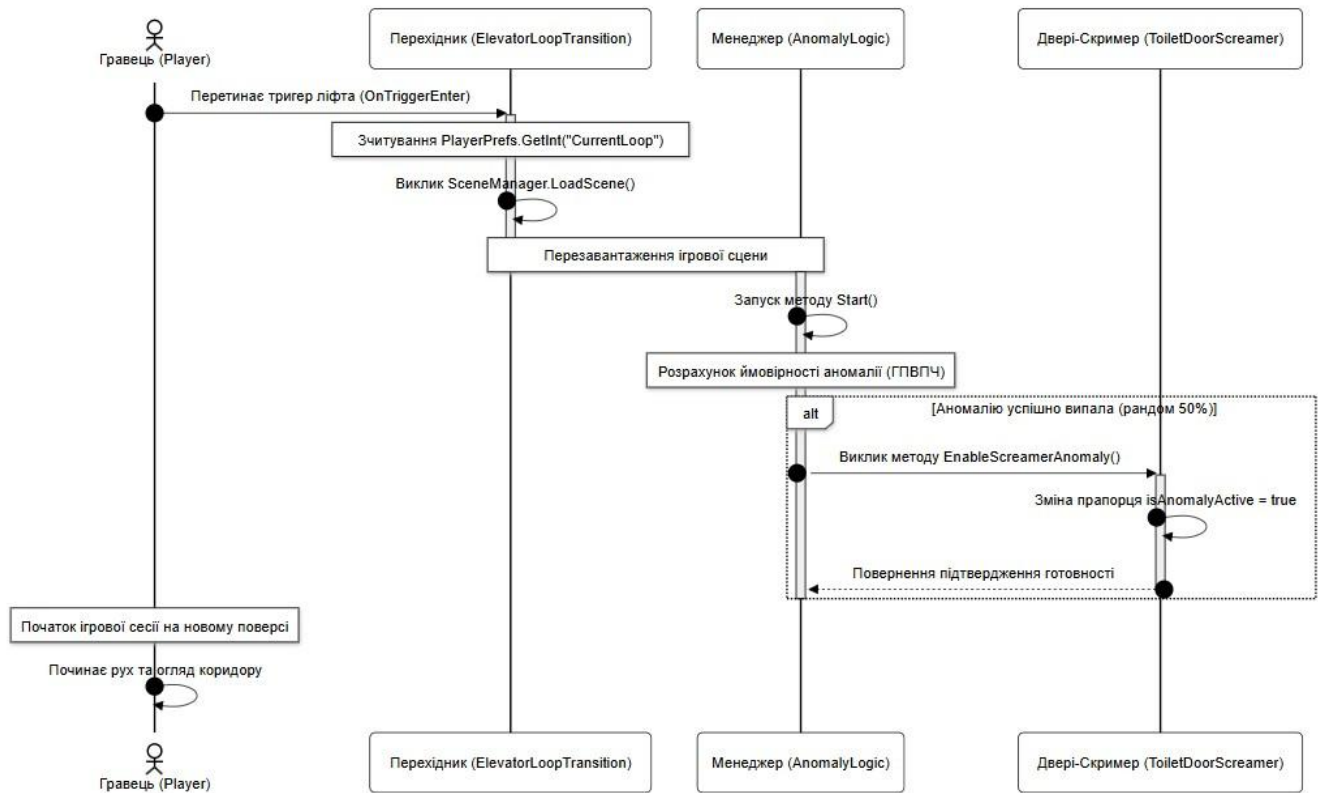
ДІАГРАМА КЛАСІВ(СИСТЕМА ГРАВЦЯ ТА ЯДРА ГРИ, ІНТЕРАКТИВНІ ОБ'ЄКТИ ТА АНОМАЛІЇ)



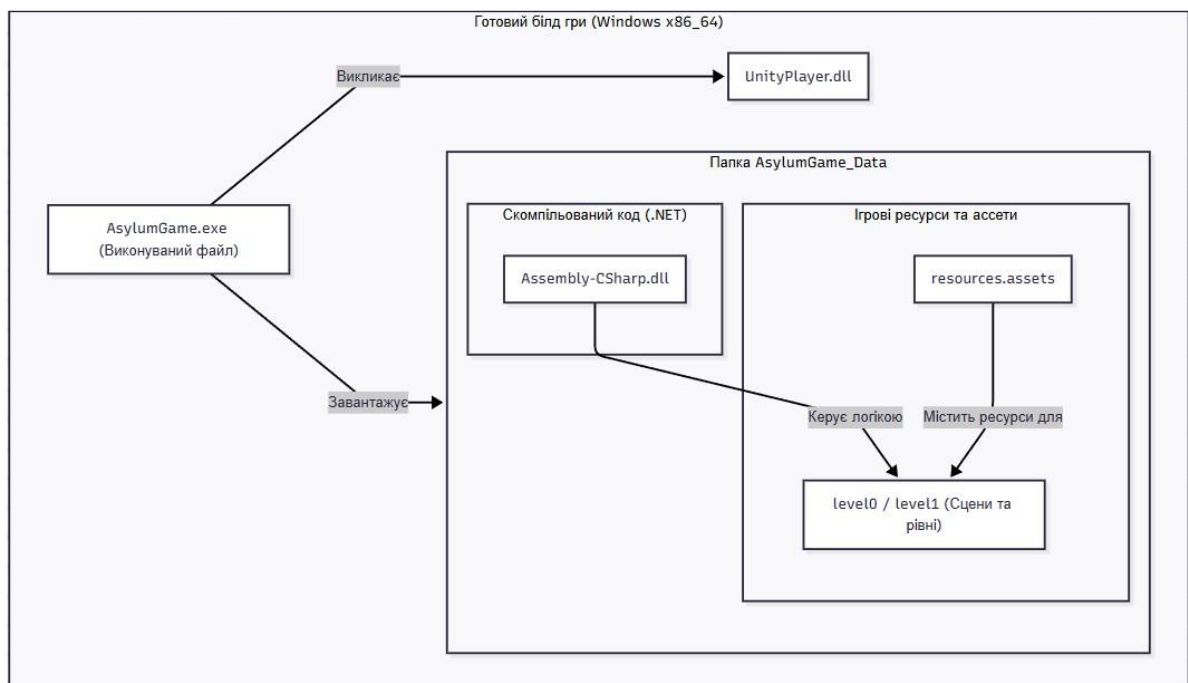
ДІАГРАМА ДІЯЛЬНОСТІ АЛГОРИТМУ ПЕРЕВІРКИ ІГРОВИХ ЦІКЛІВ



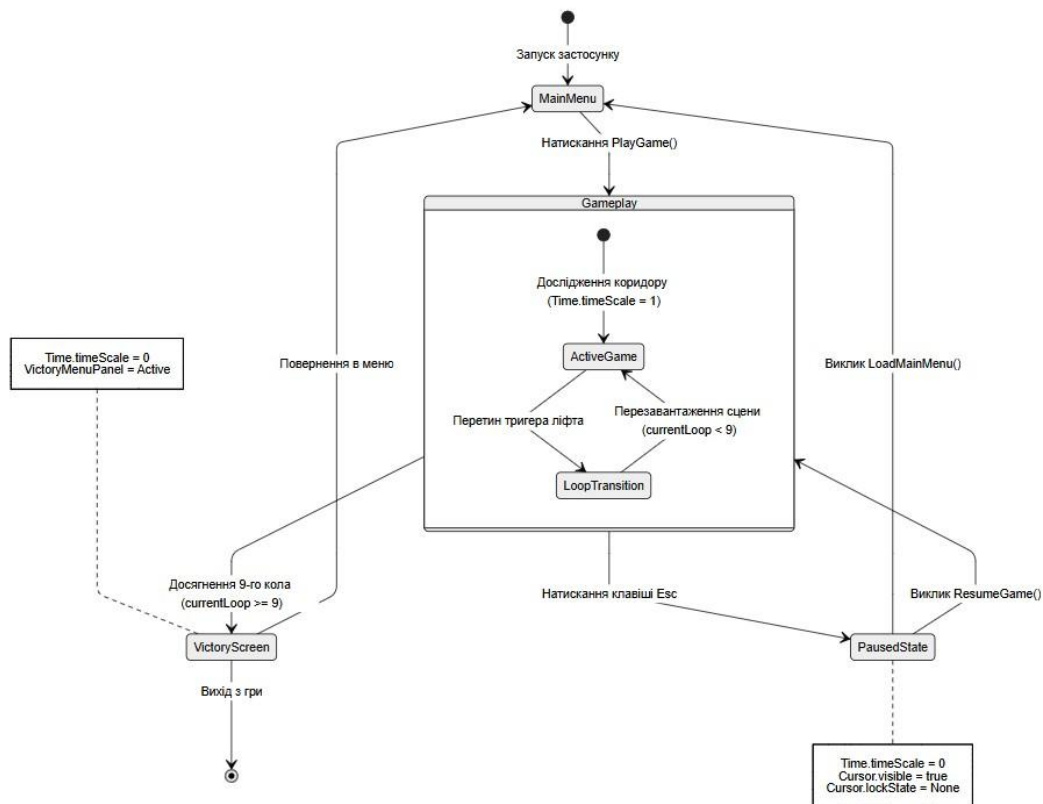
ДІАГРАМА ПОСЛІДОВНОСТІ ЗАВАНТАЖЕННЯ АНОМАЛІЙ



ДІАГРАМА КОМПОНЕНТІВ СКОМПІЛЬОВАНОГО БІЛДУ ІГРОВОГО ДОДАТКУ



ДІАГРАМА СТАНІВ СХЕМА АЛГОРИТМУ ПЕРЕВІРКИ УМОВ ПРОХОДЖЕННЯ КОРИДОРУ



ЕКРАННІ ФОРМИ





ВИСНОВКИ

У курсовій роботі було успішно вирішено актуальну інженерно -технічну задачу з проектування, розробки та тестування геймплейного вертикального зрізу тривимірної комп'ютерної гри жанру психологічного хоррору на базі сучасного ігрового рушія Unity та високорівневої мови програмування C#.

Проаналізовано предметну область та специфіку побудови ігор у сетингу лімінальних просторів. На основі аналізу класичних представників жанру було сформовано вимоги до побудови атмосфери, логіки взаємодії з оточенням, механіки циклічних переходів поверхів та генерації випадкових аномалій.

Розроблений програмний продукт повністю відповідає поставленим технічним вимогам, демонструє стабільну частоту кадрів (FPS) та високу інженерну якість коду, маючи потенціал для подальшого масштабування та публікації на платформі Steam.

ДОДАТОК Б. РЕПОЗИТОРІЙ ПРОГРАМНИХ МОДУЛІВ

<https://github.com/Rademt/My-project-1->

ДОДАТОК В. ЛІСТИНГИ ПРОГРАМНИХ МОДУЛІВ

Менеджер Аномалій

```
using UnityEngine;
using System.Collections.Generic;

public class AnomalyLogic : MonoBehaviour
{
    public enum AnomalyType { SwapObjects, JustSpawn, ScaleObject, TriggerScreamer }

    [System.Serializable]
    public class AdvancedAnomaly
    {
        public string name;
        public AnomalyType type;

        [Header("For Swapping / Spawning")]
        public GameObject normalObject;
        public GameObject anomalyObject;

        [Header("For Scaling")]
        public GameObject objectToScale;
        public Vector3 targetScale = new Vector3(2f, 2f, 2f);

        [Header("For Custom Scripts (like Screamers)")]
        public GameObject screamerTriggerObject;
    }

    [Tooltip("Шанс появи аномалії (0-100)")]
    public int AnomalyChance = 50;

    [Header("🔧 РЕЖИМ ТЕСТУВАННЯ (ДЛЯ РОЗРОБНИКА)")]
    [Tooltip("Якщо включено, рандом відключається, і завжди спавниться аномалія, вибрана нижче")]
    public bool debugMode = false;
    [Tooltip("Індекс аномалії зі списку нижче, котру треба примусово увімкнути (0, 1, 2...)")]
    public int debugAnomalyIndex = 0;

    [Header("Список усіх аномалій")]
    public List<AdvancedAnomaly> allAnomalies;

    [HideInInspector]
    public bool wasAnomalySpawned = false;

    void Start()
    {
        Debug.Log($"[DEBUG] Головний тригер '{gameObject.name}' успішно активовано на сцені й готовий ловити гравця!");
    }
}

#if UNITY_EDITOR
```

```

if (Time.realtimeSinceStartup < 2f)
{
    PlayerPrefs.SetInt("CurrentLoop", debugMode ? 1 : 0);
    PlayerPrefs.Save();
    Debug.Log("<color=orange>[РЕДАКТОР]: Налаштування поверхів оптимізовано для тестування!</color>");
}
#endif

wasAnomalySpawned = false;
if (allAnomalies == null || allAnomalies.Count == 0) return;

foreach (var anomaly in allAnomalies)
{
    if (anomaly.anomalyObject != null) anomaly.anomalyObject.SetActive(false);
    if (anomaly.normalObject != null) anomaly.normalObject.SetActive(true);

    if (anomaly.screamerTriggerObject != null)
    {
        var wheelScript = anomaly.screamerTriggerObject.GetComponentInChildren<TriggerScreamerObject>();

        if (wheelScript != null)
        {
            anomaly.screamerTriggerObject.SetActive(false);
        }
        else
        {
            var toiletScript = anomaly.screamerTriggerObject.GetComponentInChildren<ToiletDoorScreamer>();
            if (toiletScript != null)
            {
                {
            }
        }
    }
}

Random.InitState(System.DateTime.Now.Millisecond + System.DateTime.Now.Second);
int currentLoop = PlayerPrefs.GetInt("CurrentLoop", 0);

if (debugMode)
{
    if (debugAnomalyIndex >= 0 && debugAnomalyIndex < allAnomalies.Count)
    {
        wasAnomalySpawned = true;
        ActivateAnomaly(allAnomalies[debugAnomalyIndex]);
        Debug.Log($"<color=magenta>[DEBUG РЕЖИМ]: Аномалія була запущена примусово
№{debugAnomalyIndex} — {allAnomalies[debugAnomalyIndex].name}!</color>");
    }
    else
    {
        Debug.LogError($"[DEBUG ПОМИЛКА]: Індекс {debugAnomalyIndex} немає у списку аномалій!");
    }
    return;
}

if (currentLoop == 0)
{
    Debug.Log("<color=yellow>[ЭТАЖ 0]:</color> Ознайомлювальне коло. Абсолютна чистота.");
    return;
}

int globalRoll = Random.Range(0, 101);

if (globalRoll <= AnomalyChance)
{

```

```

        wasAnomalySpawned = true;
        int randomIndex = Random.Range(0, allAnomalies.Count);
        ActivateAnomaly(allAnomalies[randomIndex]);
    }
    else
    {
        Debug.Log($"<color=green>[ЧИСТЕ КОЛО]:</color> Рол не пройшов. Колісниця спить. Поверх:
{currentLoop}");
    }
}

private void ActivateAnomaly(AdvancedAnomaly anomaly)
{
    switch (anomaly.type)
    {
        case AnomalyType.SwapObjects:
            if (anomaly.normalObject != null) anomaly.normalObject.SetActive(false);
            if (anomaly.anomalyObject != null) anomaly.anomalyObject.SetActive(true);
            break;

        case AnomalyType.JustSpawn:
            if (anomaly.anomalyObject != null) anomaly.anomalyObject.SetActive(true);
            break;

        case AnomalyType.ScaleObject:
            if (anomaly.objectToScale != null) anomaly.objectToScale.transform.localScale = anomaly.targetScale;
            break;

        case AnomalyType.TriggerScreamer:
            if (anomaly.screamerTriggerObject != null)
            {
                anomaly.screamerTriggerObject.SetActive(true);

                ToiletDoorScreamer toiletScript =
anomaly.screamerTriggerObject.GetComponentInChildren<ToiletDoorScreamer>();
                if (toiletScript != null)
                {
                    toiletScript.EnableScreamerAnomaly();
                }

                TriggerScreamerObject wheelScript =
anomaly.screamerTriggerObject.GetComponentInChildren<TriggerScreamerObject>();
                if (wheelScript != null)
                {
                    Debug.Log($"[AnomalyLogic] Колісниця '{anomaly.screamerTriggerObject.name}' активована як
АНОМАЛІЯ!");
                }
            }
            break;
    }

    Debug.Log($"<color=cyan>[ВИБРАНО АНОМАЛІЮ]:</color> {anomaly.name}");
}
}

```

Менеджер переходу крізь поверхи

```

using UnityEngine;
using System.Collections.Generic;
using UnityEngine.SceneManagement;

public class ElevatorLoopTransition : MonoBehaviour

```

```

{
    [Header("Налаштування дверей")]
    public bool isCorrectChoice;

    [Header("Компоненти ліфта")]
    public Animator elevatorAnimator;
    public string closeAnimationName = "Elevator_Close";

    [Header("Звуковий супровід")]
    public AudioSource doorOpenSound;
    public AudioSource doorCloseSound;
    public AudioSource elevatorMusic;

    [Header("Логіка аномалій")]
    public AnomalyLogic anomalyLogic;

    [Header("Екран Перемоги (UI)")]
    public GameObject victoryMenuPanel;

    [Header("Скрипти для вимкнення після перемоги")]
    public GameObject pauseControllerObject;
    public MonoBehaviour playerMovementScript;

    private bool playerInside = false;

    void Start()
    {
        Time.timeScale = 1f;

        if (doorOpenSound != null)
        {
            doorOpenSound.Play();
        }
    }

    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player") && !playerInside)
        {
            playerInside = true;
            StartElevatorProcess();
        }
    }

    void StartElevatorProcess()
    {
        if (elevatorAnimator != null)
        {
            elevatorAnimator.Play(closeAnimationName);
        }

        if (doorCloseSound != null)
        {
            doorCloseSound.Play();
        }
    }
}

```

```

    Debug.Log("Двері ліфта зачиняються...");

    Invoke("PlayElevatorMusic", 2.5f);

    Invoke("CompleteLoopTransition", 7f);
}

void PlayElevatorMusic()
{
    if (elevatorMusic != null)
    {
        elevatorMusic.Play();
        Debug.Log("Ліфт рушив. Лунає музика супроводу.");
    }
}

void CompleteLoopTransition()
{
    if (elevatorMusic != null) elevatorMusic.Stop();

    int currentLoop = PlayerPrefs.GetInt("CurrentLoop", 0);

    bool thereWasAnomaly = false;
    if (anomalyLogic != null)
    {
        thereWasAnomaly = anomalyLogic.wasAnomalySpawned;
    }

    bool playerGuessedRight = false;

    if (thereWasAnomaly)
    {
        playerGuessedRight = !isCorrectChoice;
    }
    else
    {
        playerGuessedRight = isCorrectChoice;
    }

    if (playerGuessedRight)
    {
        currentLoop++;
        PlayerPrefs.SetInt("CurrentLoop", currentLoop);
        PlayerPrefs.Save();
        Debug.Log($"Правильно! Ліфт приїхав на поверх: {currentLoop}");

        if (currentLoop >= 3)
        {
            Debug.Log("ПЕРЕМОГА!");
            PlayerPrefs.SetInt("CurrentLoop", 0);
            PlayerPrefs.Save();

            TriggerVictoryWindow();
            return;
        }
    }
}

```

```

    }
}
else
{
    Debug.Log("Помилка! Ліфт повертає вас на самий початок.");
    PlayerPrefs.SetInt("CurrentLoop", 0);
    PlayerPrefs.Save();
}

SceneManager.LoadScene(SceneManager.GetActiveScene().name);
}

private void TriggerVictoryWindow()
{
    if (pauseControllerObject != null)
    {
        pauseControllerObject.SetActive(false);
    }

    if (playerMovementScript != null)
    {
        playerMovementScript.enabled = false;
    }

    if (victoryMenuPanel != null)
    {
        victoryMenuPanel.SetActive(true);
    }

    Time.timeScale = 0f;

    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}

public void LoadMainMenu(string sceneName)
{
    Time.timeScale = 1f;
    SceneManager.LoadScene(sceneName);
}
}

```

Менеджер зміни цифри поверху

```

using UnityEngine;
using TMPro;

public class FloorSignUpdater : MonoBehaviour
{
    private TextMeshPro textMesh3D;
    private TextMeshProUGUI textMeshUI;

    void Start()
    {
        UpdateFloorText();
    }
}

```



```

public void UpdateFloorText()
{
    int currentLoop = PlayerPrefs.GetInt("CurrentLoop", 0);
    Debug.Log($"[FloorSignUpdater] Вирівнюю рівень на стіні: {currentLoop}");

    textMesh3D = GetComponent<TextMeshPro>();
    if (textMesh3D != null)
    {
        textMesh3D.text = currentLoop.ToString();
        return;
    }

    textMeshUI = GetComponent<TextMeshProUGUI>();
    if (textMeshUI != null)
    {
        textMeshUI.text = currentLoop.ToString();
    }
}

[ContextMenu("Повернути гру на 0 поверх")]
public void ResetPrefsForDebug()
{
    PlayerPrefs.SetInt("CurrentLoop", 0);
    PlayerPrefs.Save();
    UpdateFloorText();
    Debug.Log("<color=red>[DEBUG]: Усі збереження видалено вручну. Ви перебуваєте на 0-му поверсі!</color>");
}
}

```

Контролер гравця

```

using UnityEngine;

namespace SojaExiles
{
    public class PlayerMovement : MonoBehaviour
    {
        public CharacterController controller;
        public Transform cameraTransform;

        public float walkSpeed = 3f;
        public float runSpeed = 6f;
        public float gravity = -25f;

        [Header("Налаштування коливання камери")]
        public float walkingBobAmount = 0.07f;
        public float walkingBobSpeed = 12f;
        public float runningBobAmount = 0.12f;
        public float runningBobSpeed = 16f;

        [Header("Звуки шагів")]
        public AudioSource stepsAudio;
        public AudioClip walkClip;
        public AudioClip runClip;
    }
}

```

```

Vector3 velocity;
bool isGrounded;
float defaultYPos;
float timer;

private float lastSinValue = 0f;

private SlimUI.ModernMenu.UISettingsManager keyManager;

void Start()
{
    if (cameraTransform != null) defaultYPos = cameraTransform.localPosition.y;

    keyManager = FindObjectOfType<SlimUI.ModernMenu.UISettingsManager>();

    if (keyManager == null)
    {
        Debug.LogError("UISettingsManager не знайшли на сцені!");
    }
}

void Update()
{
    isGrounded = controller.isGrounded;

    if (isGrounded && velocity.y < 0)
    {
        velocity.y = -2f;
    }

    bool isRunning = Input.GetKey(KeyCode.LeftShift);
    float x = Input.GetAxisRaw("Horizontal");
    float z = Input.GetAxisRaw("Vertical");

    if (keyManager != null && keyManager.keys.Count > 0)
    {
        isRunning = Input.GetKey(keyManager.keys["Sprint"]);
        x = 0f;
        z = 0f;

        if (Input.GetKey(keyManager.keys["Forward"])) z += 1f;
        if (Input.GetKey(keyManager.keys["Backward"])) z -= 1f;
        if (Input.GetKey(keyManager.keys["Left"])) x -= 1f;
        if (Input.GetKey(keyManager.keys["Right"])) x += 1f;
    }

    float currentSpeed = isRunning ? runSpeed : walkSpeed;

    Vector3 move = transform.right * x + transform.forward * z;
    if (move.magnitude > 1) move.Normalize();

    controller.Move(move * currentSpeed * Time.deltaTime);

    if (isGrounded && move.magnitude > 0.1f && cameraTransform != null)

```

```

{
    float speedMultiplier = isRunning ? runningBobSpeed : walkingBobSpeed;
    timer += Time.deltaTime * speedMultiplier;
    float amount = isRunning ? runningBobAmount : walkingBobAmount;

    float sinValue = Mathf.Sin(timer);
    float newY = defaultYPos + sinValue * amount;
    cameraTransform.localPosition = new Vector3(cameraTransform.localPosition.x, newY,
cameraTransform.localPosition.z);
    if (sinValue < -0.95f && lastSinValue >= -0.95f)
    {
        if (stepsAudio != null)
        {
            AudioClip clipToPlay = isRunning ? runClip : walkClip;
            if (clipToPlay != null && (!stepsAudio.isPlaying || stepsAudio.clip != clipToPlay))
            {
                stepsAudio.clip = clipToPlay;

                stepsAudio.pitch = Random.Range(0.9f, 1.1f);
                stepsAudio.volume = isRunning ? 0.8f : 0.45f;
                stepsAudio.Play();
            }
        }
    }
    lastSinValue = sinValue;
}
else if (cameraTransform != null)
{
    timer = 0;
    lastSinValue = 0f;
    cameraTransform.localPosition = new Vector3(
        cameraTransform.localPosition.x,
        Mathf.Lerp(cameraTransform.localPosition.y, defaultYPos, Time.deltaTime * 5f),
        cameraTransform.localPosition.z
    );
}
velocity.y += gravity * Time.deltaTime;
controller.Move(velocity * Time.deltaTime);
}
}
}

```